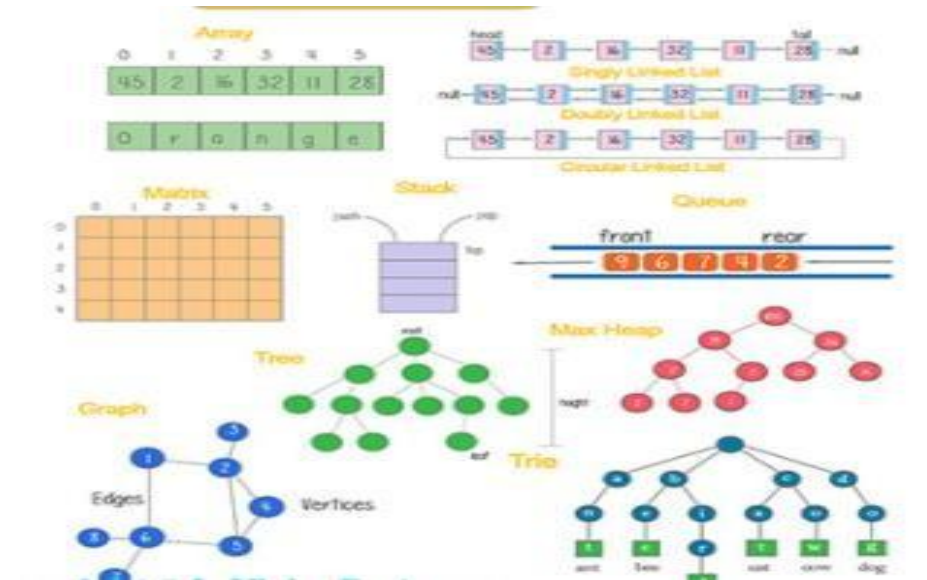


Data Structures



Dr. Elakkiya

Asst. Professor,

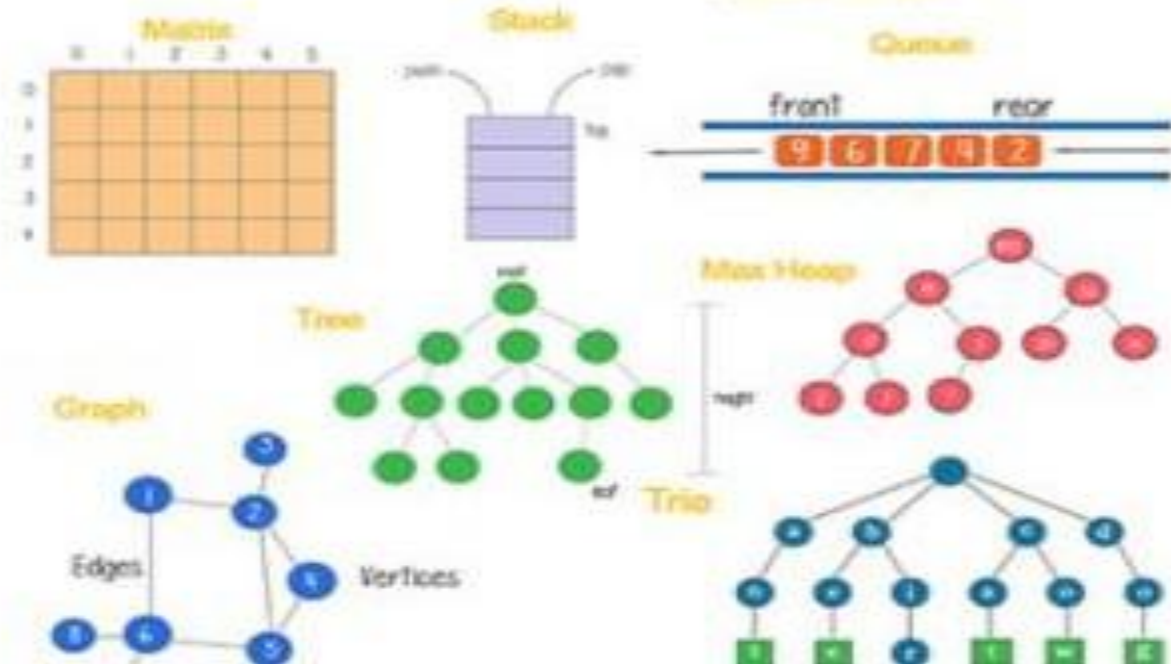
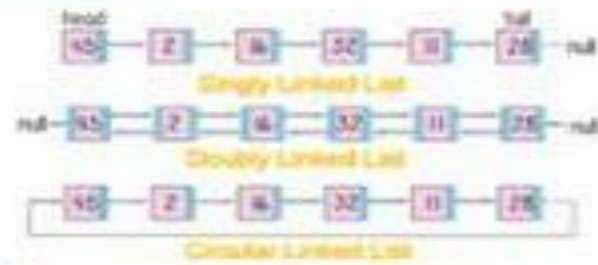
Dept. of CSE, SRM University AP



SRM
UNIVERSITY AP

Andhra Pradesh

Data structures



TREES

UNIT-III

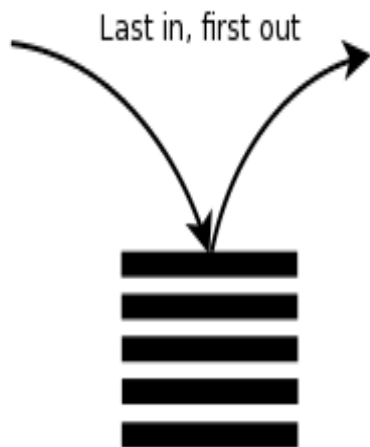
Introduction to Trees

Why Tree data structure?

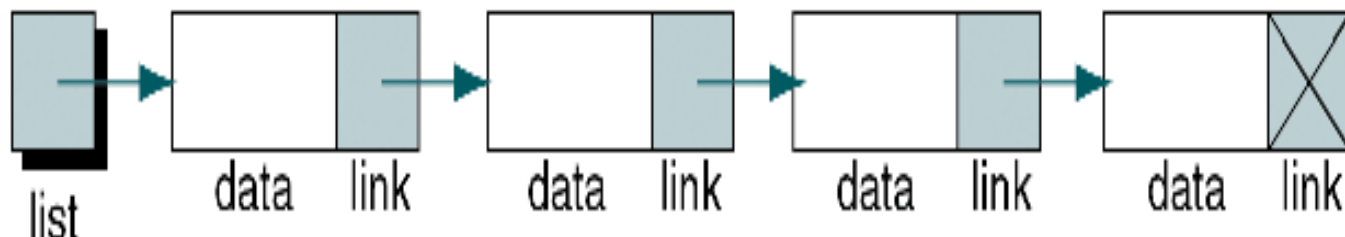
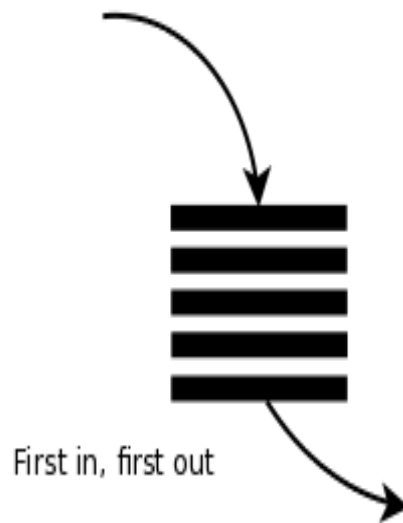
Array

17	23	97	44	33			
0	1	2	3	4	5	6	7

Stack:



Queue:



▶ Linear data structure:

- Stores the data one after the other (for an element we can find before element and after element).

▶ Choosing an appropriate data structure:

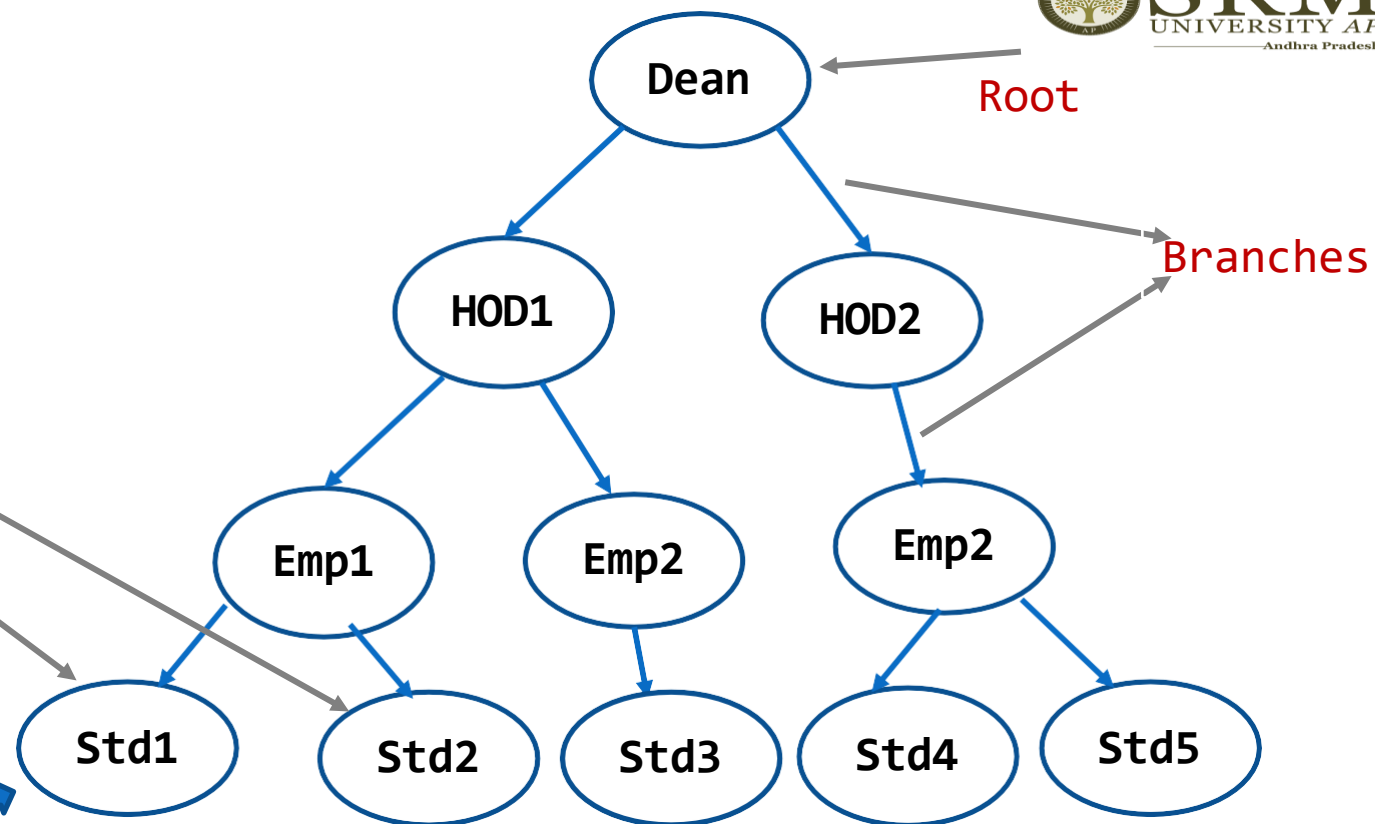
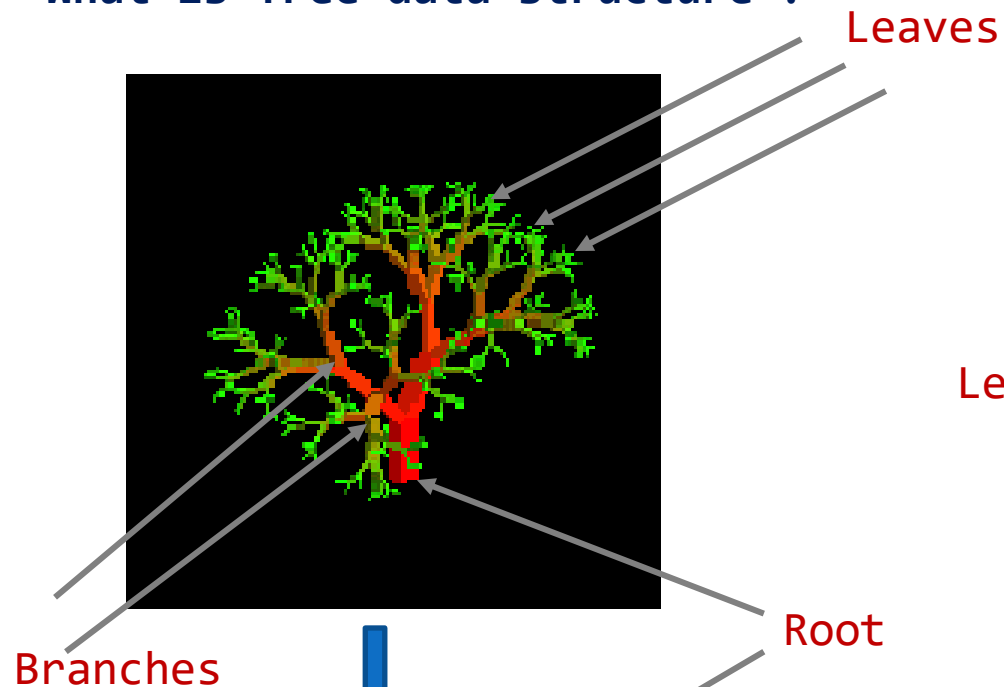
- Best suited for problem scenario
- Cost of common operations
- Space utilization

▶ Tree data structure used to:

- Store the data hierarchical (Ex: File System in an Operating System)
- Organize the data for quick insertion, deletion and search (Ex: Binary Search Algorithm)
- Solve complex problems (Ex: Network routing Algorithms)

Introduction to Trees

What is Tree data structure ?

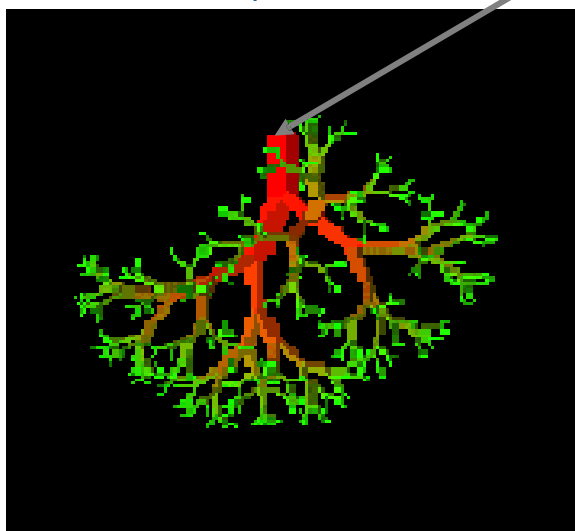


Logical representation of Tree data structure

Tree data structure is an efficient way of storing and organizing the data that is naturally hierarchical.

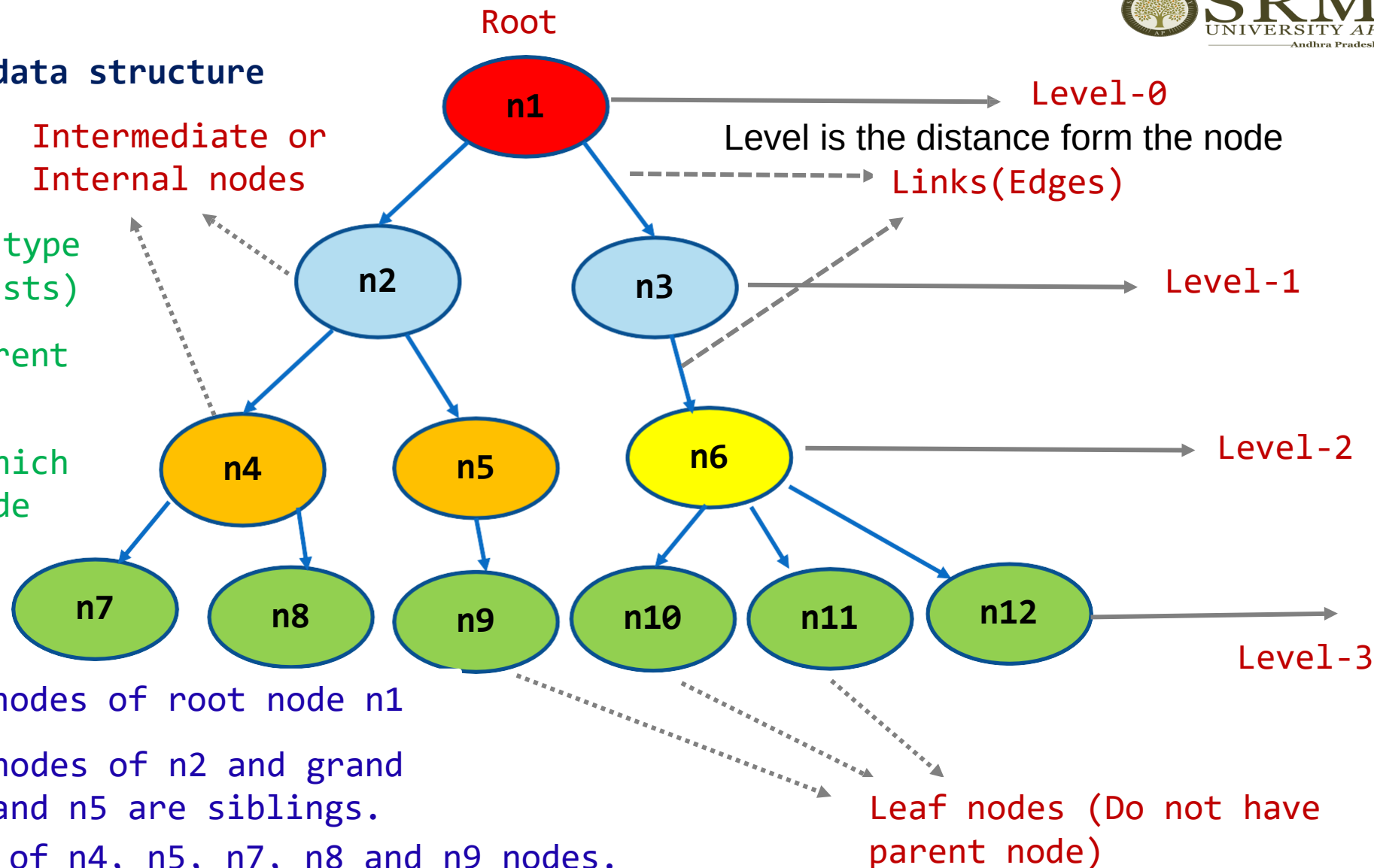
OR

Tree data structure is a non-linear (hierarchical) data structure that consists of nodes with parent-child relationship.



Terminology used in Tree data structure

- ▶ A node can contain any type of data (Like Linked lists)
- ▶ The root (n1) is the parent of n2 and n3 nodes
- ▶ Root is the only node which does not have parent node

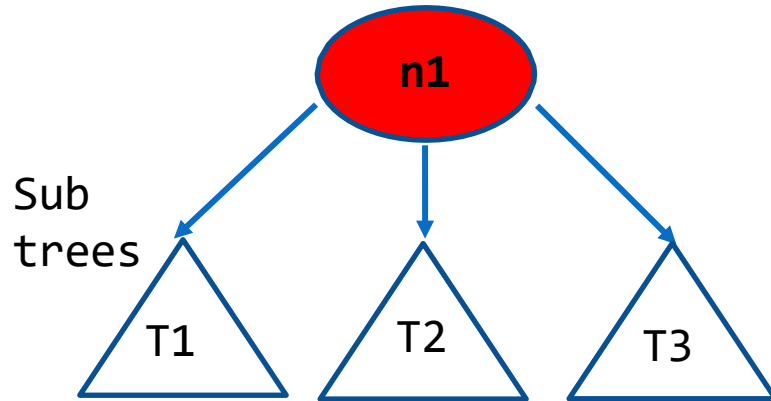


- ▶ n2 and n3 are children nodes of root node n1
- ▶ n4 and n5 are children nodes of n2 and grand children of n1. And n4 and n5 are siblings.
- ▶ n1 and n2 are ancestors of n4, n5, n7, n8 and n9 nodes.
- ▶ n4 and n5 are descendants of n2 node.
- ▶ Nodes not having same parent with same grand parent are called cousins (Ex: n8 and n9)

Introduction to Trees

Properties of Tree data structure

1. Tree is a recursive data structure



1. **Degree** The number of branches associated with a node
2. **In-degree** a node is the no. of edges arriving at that node.
4. **Out-degree** of a node is the no. of edges leaving that node.

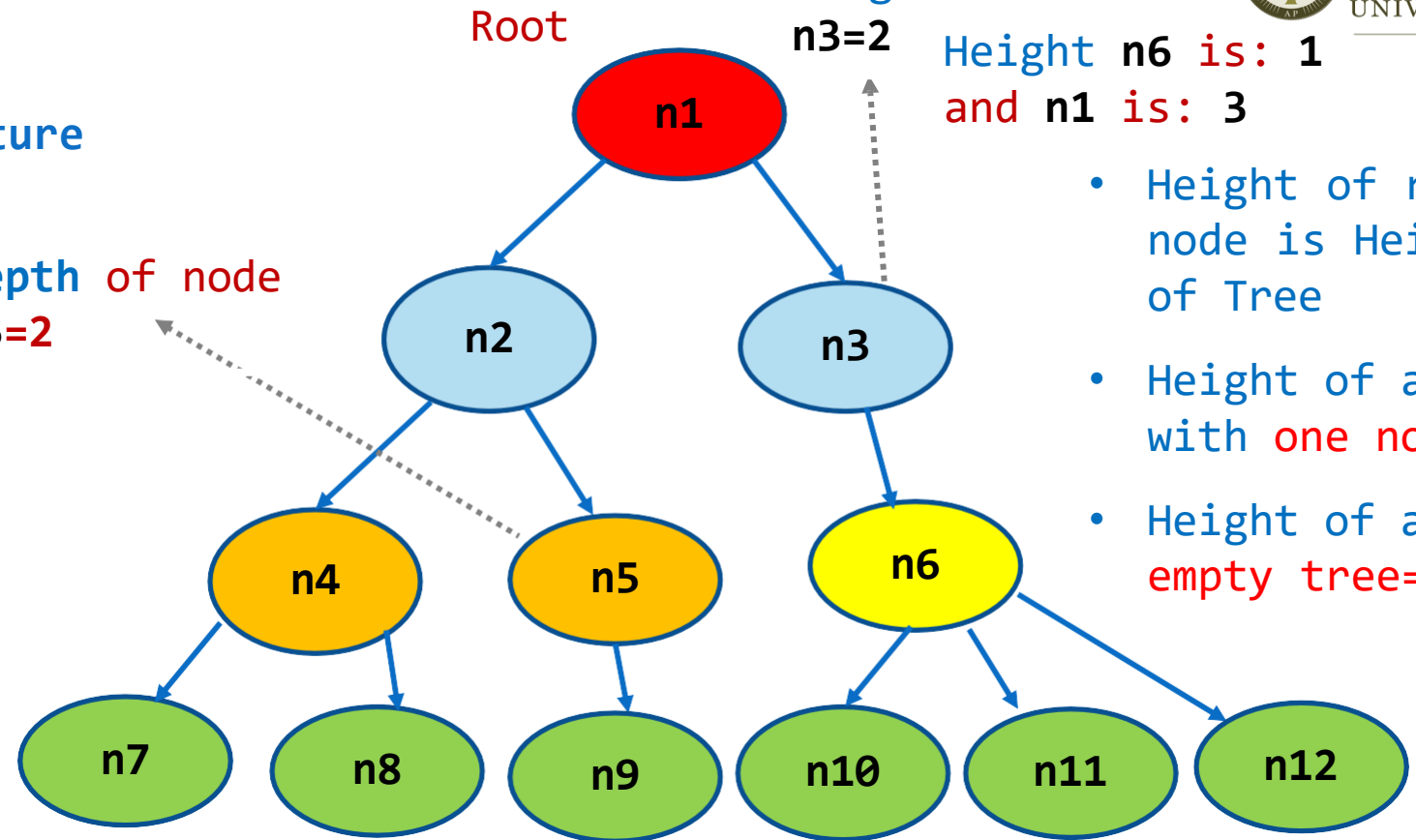
5. If Tree contains n nodes then it contains $n-1$ edges

(A tree does not have cycles). One incoming edge for each node except root.

6. Depth of a node X is length (number of edges) of the path from the root node to X

7. Height of a node X is number of edges in longest path from the node X to a leaf node

Depth of node
 $n5=2$

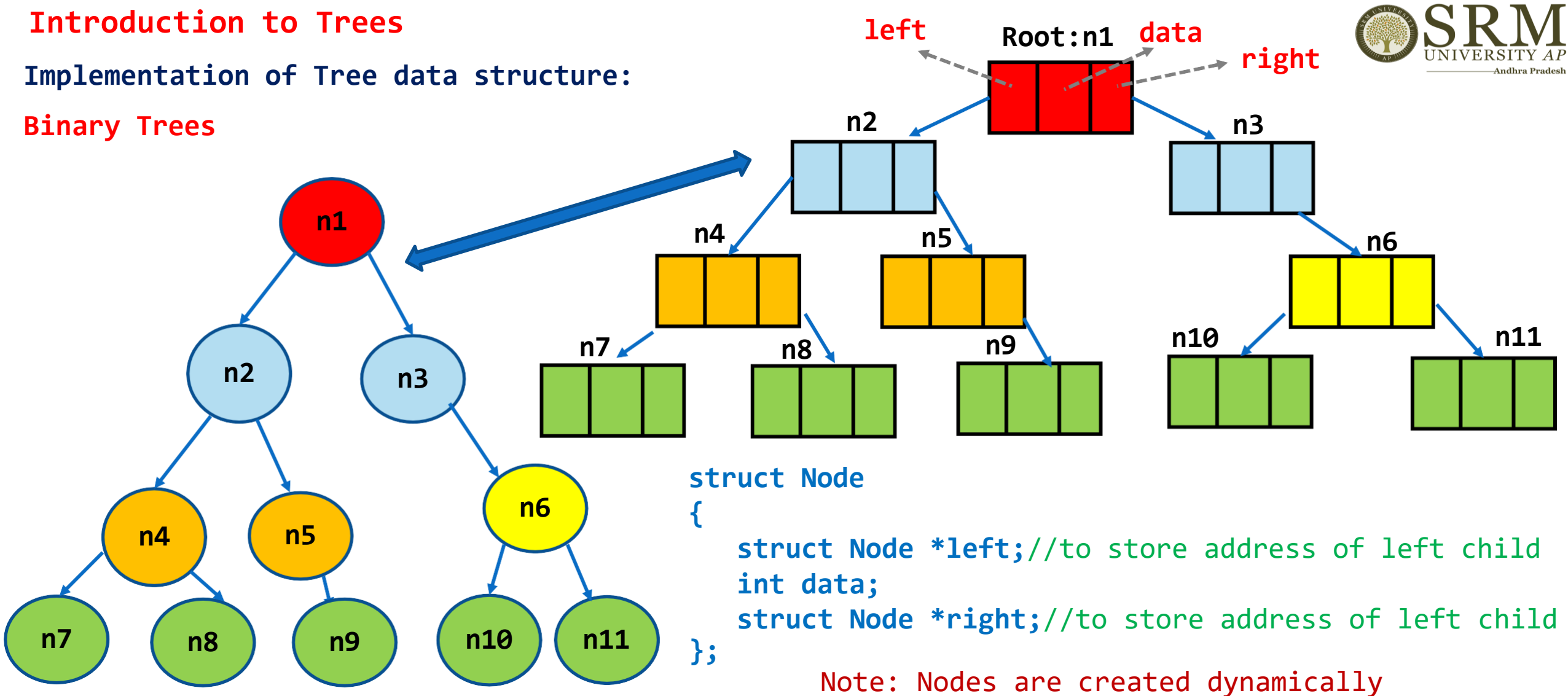


- Height of root node is Height of Tree
- Height of a tree with one node = 0
- Height of an empty tree = -1

Introduction to Trees

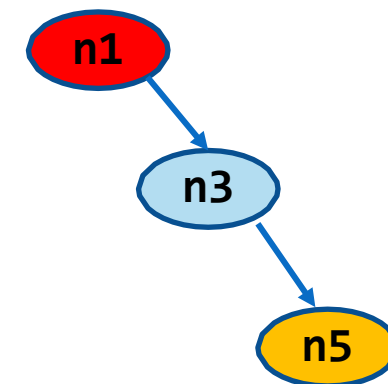
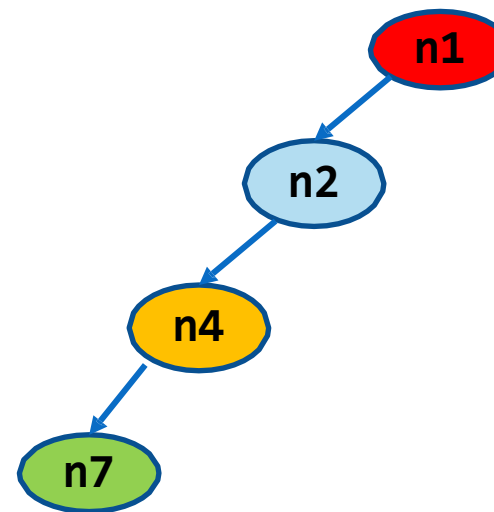
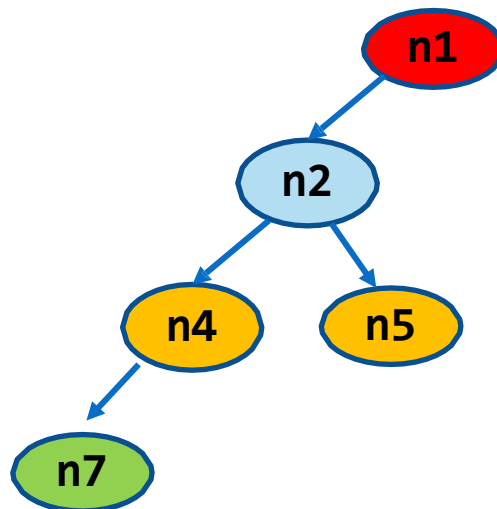
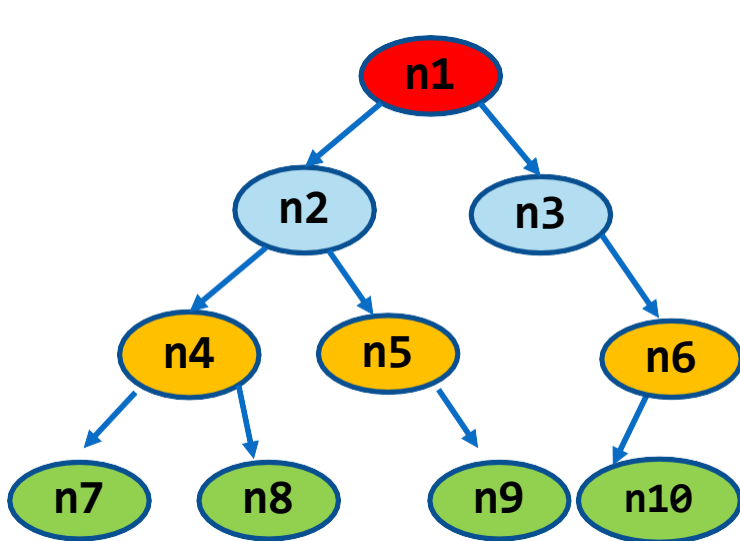
Implementation of Tree data structure:

Binary Trees

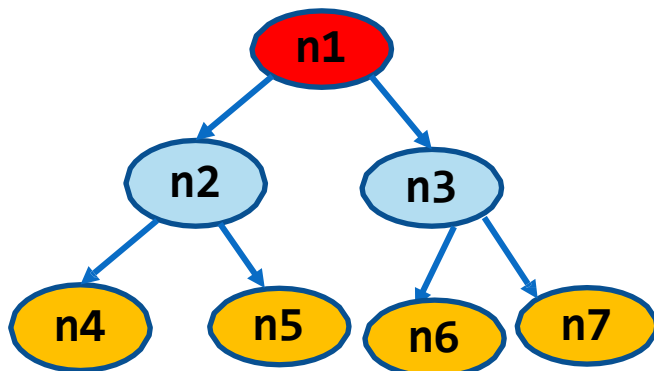


- A tree in which each node has at most two children is called as **Binary Tree**.
(No. of Children ≤ 2)

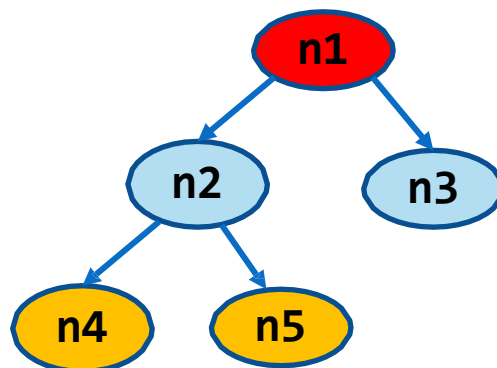
Binary Trees



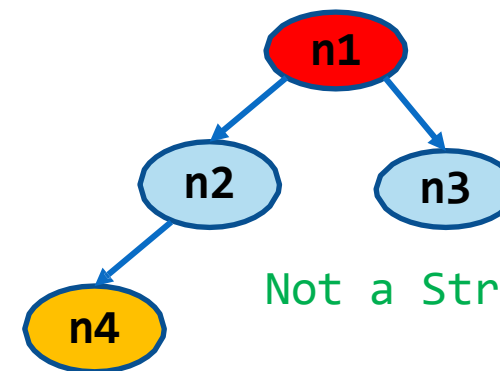
Strict Binary Tree or Proper or Full Binary Tree: In Strict Binary Tree, each node can have either zero or 2 nodes



Strict Binary Tree



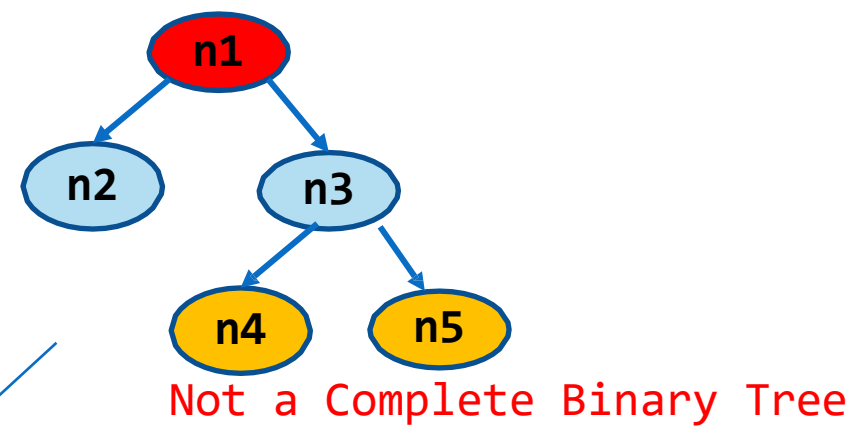
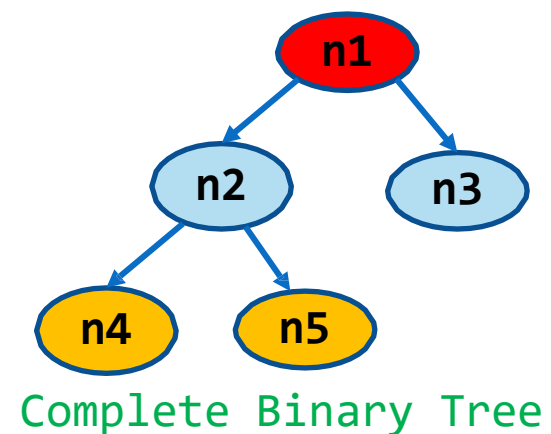
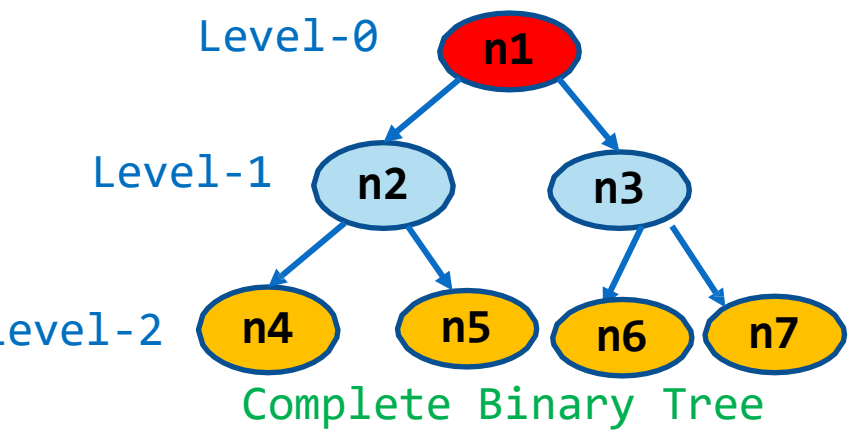
Strict Binary Tree



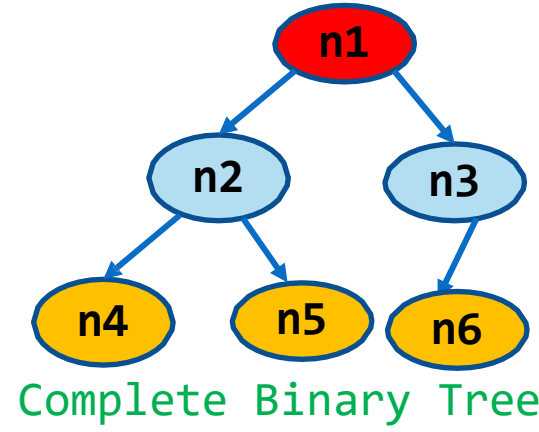
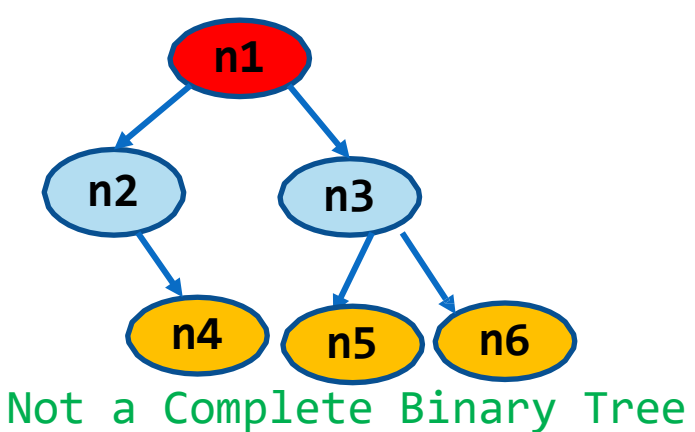
Not a Strict Binary Tree

Complete Binary Trees

In Complete Binary Tree, all the levels except possibly last are filled and all nodes are as left as possible.



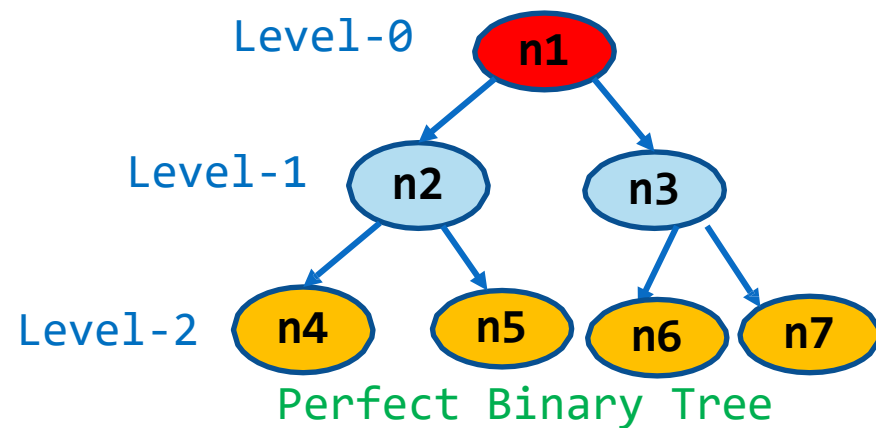
Nodes are missing here
(two vacant positions)



Maximum number of nodes in complete Binary tree at level-i = 2^i

Perfect Binary Trees

In Perfect Binary Tree, all the levels are filled.



With “n” nodes in Perfect binary tree, what is the height (h) of the tree?

Maximum no. of nodes in a binary tree with the height “h”,

$$= 2^0 + 2^1 + \dots + 2^h$$

$$= 2^{h+1} - 1$$

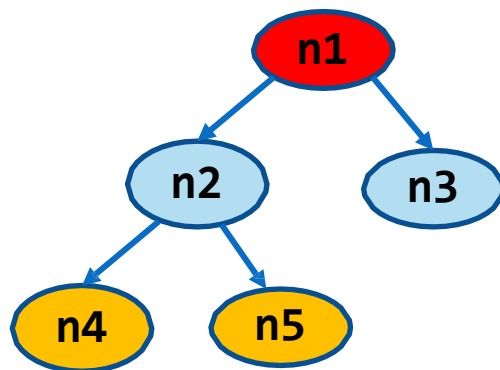
$$n = 2^{h+1} - 1$$

$$n+1 = 2^{h+1}$$

$$\log(n+1) = \log(2^{h+1})$$

$$\log(n+1) = (h+1)$$

$$h = \log(n+1) - 1$$



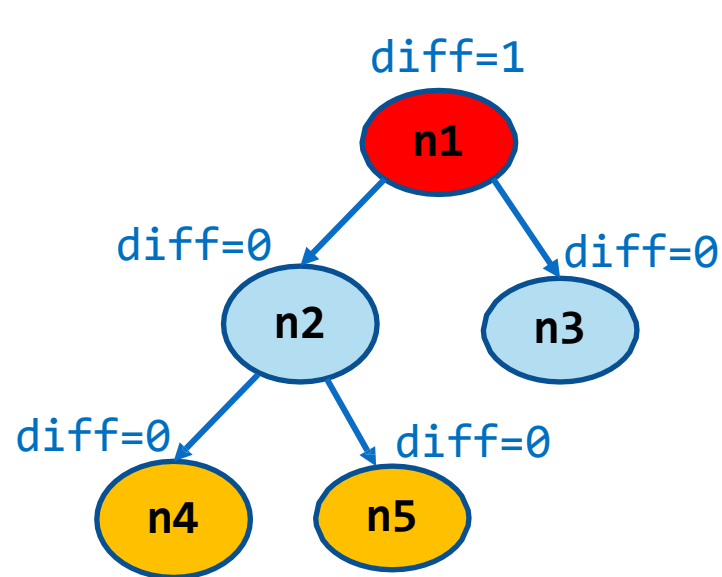
Complete Binary Tree
but not Perfect

Balanced Binary Trees of Height balanced binary trees

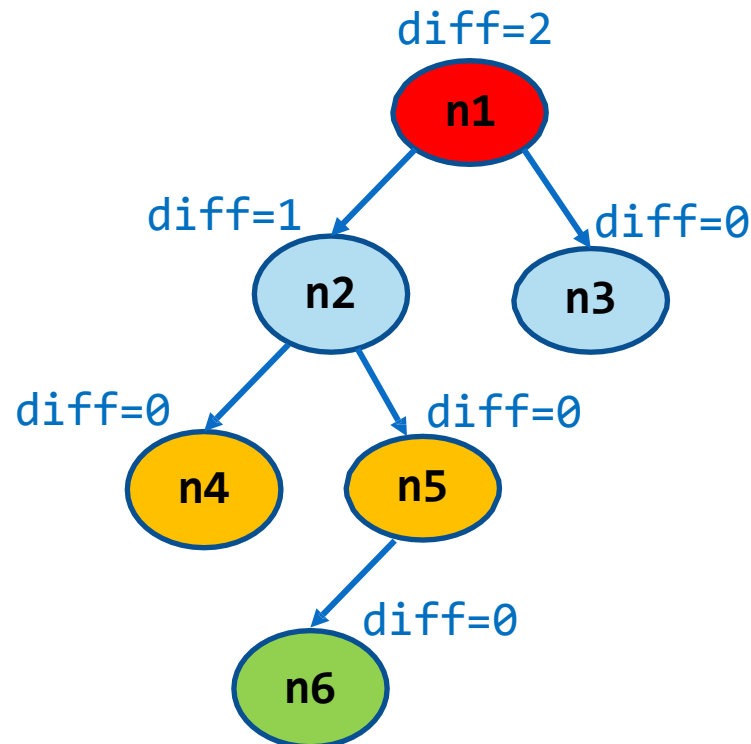
A Binary tree said to be if:

1. The height difference between the left and the right subtree for any node is not more than K (usually **K=1**).
2. The left subtree is balanced.
3. The right subtree is balanced.

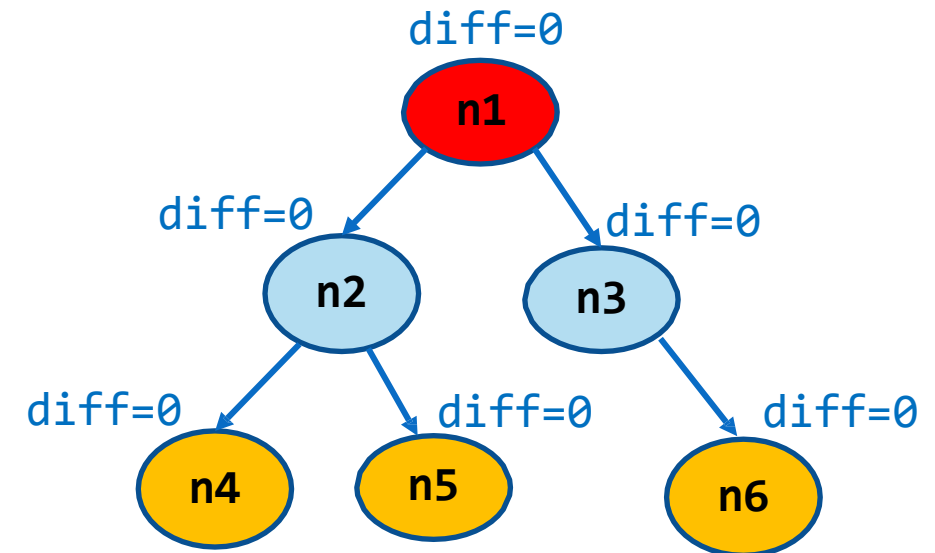
$$\text{Height diff} = | \text{Height of left child} - \text{Height of right child} |$$



Balanced binary tree



Not a Balanced binary tree

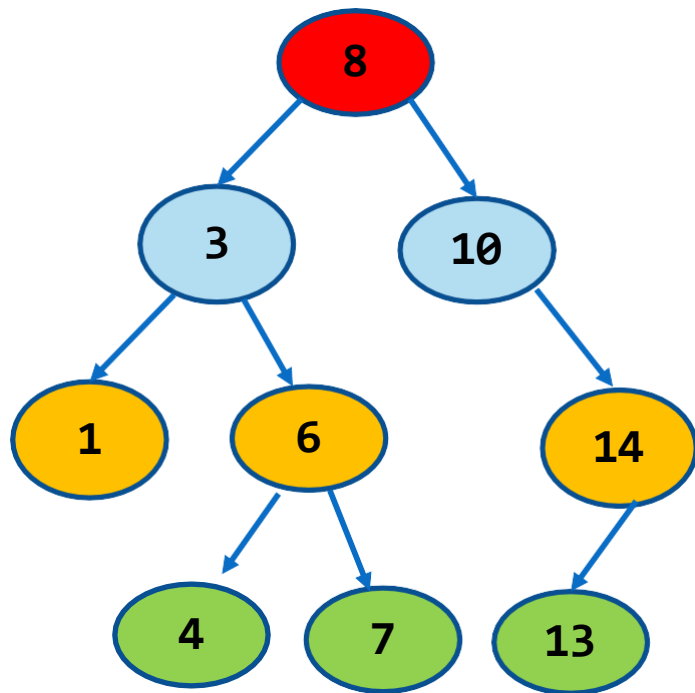


Balanced binary tree

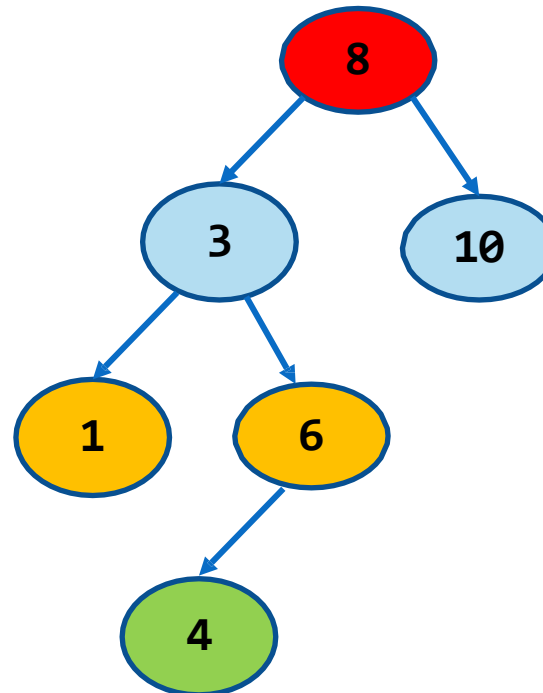
Binary Search Trees (BST)

BST is a binary tree where **each node n** satisfy the following:

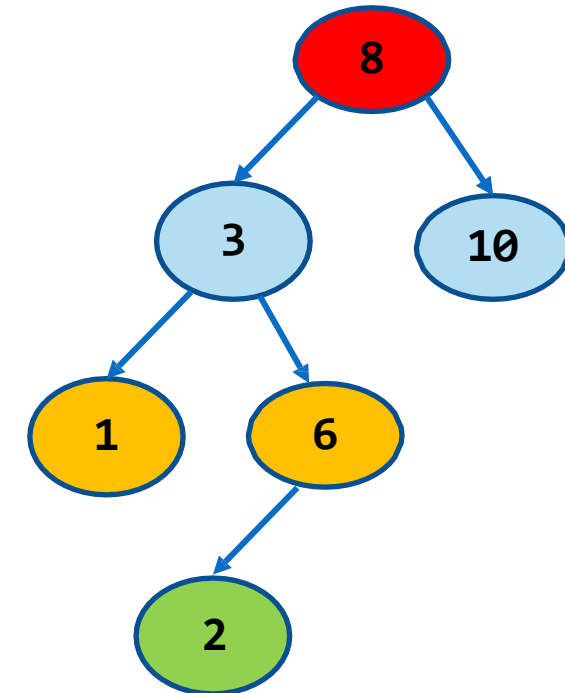
- Every node in the **left subtree of n** contains a value which is **smaller** than the value in **n**.
- Every node in the **right subtree of n** contains a value which is **larger** than the value in **n**.



Binary Search Tree



Binary Search Tree



Not a Binary Search Tree

Binary Search Trees (BST)

- ▶ Empty BST is a single pointer with the value of NULL.

```
root = NULL;
```

- ▶ A node in BST can be declared as:

```
struct Node
```

```
{
```

```
    struct Node *leftChild; //to store address of left child
```

```
    int data;
```

```
    struct Node *rightChild; //to store address of left child
```

```
};
```

- ▶ Create a node of BST as:

```
struct Node *newNode(int item)
```

```
{
```

```
    struct node *newnode = (struct Node*) malloc(sizeof(struct Node));
```

```
    newnode->data = item;
```

```
    newnode->leftChild = NULL;
```

```
    newnode->rightChild = NULL;
```

```
    return newnode;
```

```
}
```

```
int main()
```

```
{
```

```
    struct node *root;
```

```
    root=NULL;
```

```
    insert(root,45);
```

```
    insert(root,15);
```

```
    insert(root,79);
```

```
    insert(root,90);
```

```
    insert(root,10);
```

```
    insert(root,55);
```

```
    insert(root,12);
```

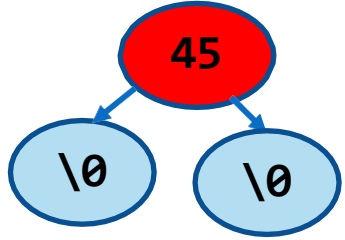
```
    insert(root,50);
```

```
}
```

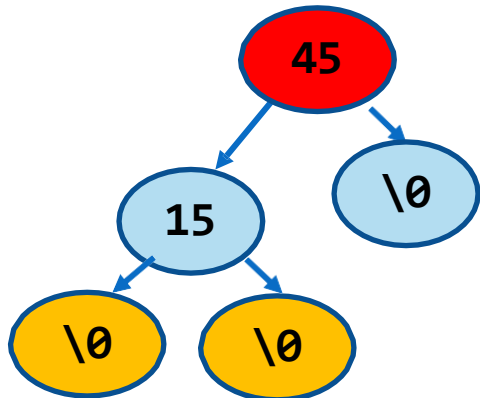
Inserting data into BST

Elements: 45, 15, 79, 90, 10, 55, 12, 50

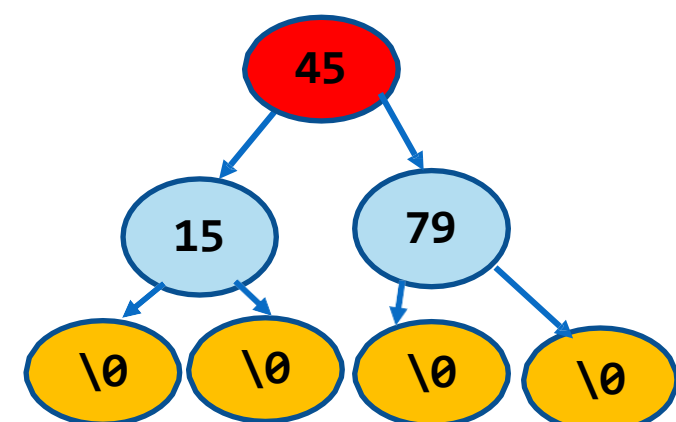
Step-1: 45 root



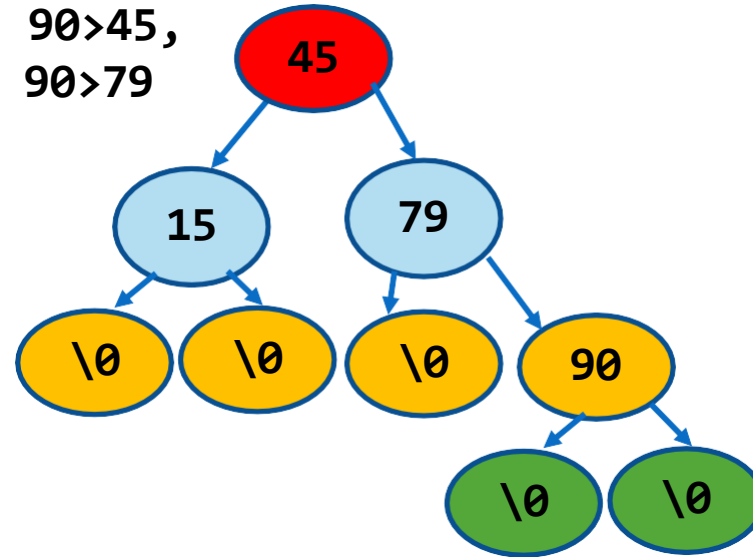
Step-2: 15, 15 < 45



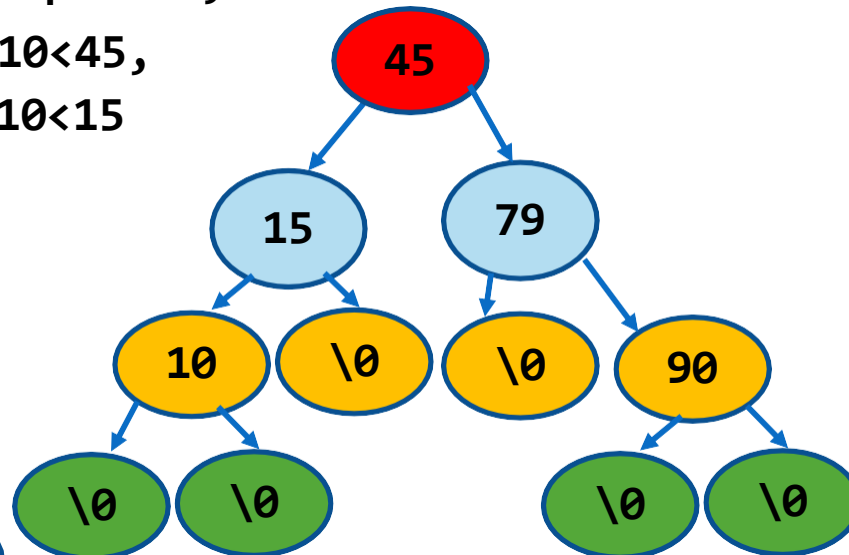
Step-3: 79, 79 > 45



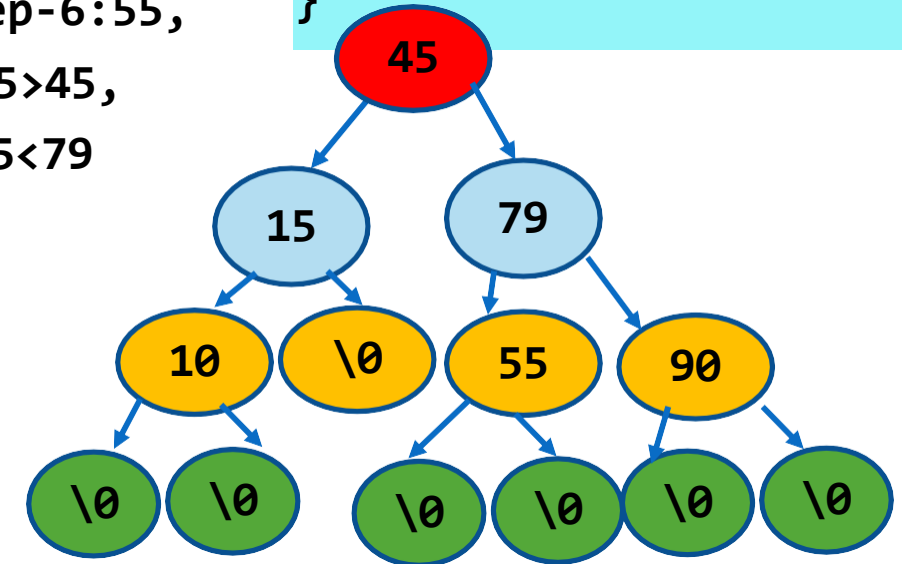
Step-4: 90,
90 > 45,
90 > 79



Step-5: 10,
10 < 45,
10 < 15



Step-6: 55,
55 > 45,
55 < 79



```

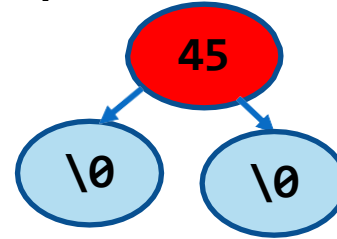
int main(){
    struct node *root;
    root=NULL;
    insert(root,45);
    insert(root,15);
    insert(root,79);
    insert(root,90);
    insert(root,10);
    insert(root,55);
    insert(root,12);
    insert(root,50);
}
  
```

Binary Search Trees (BST)

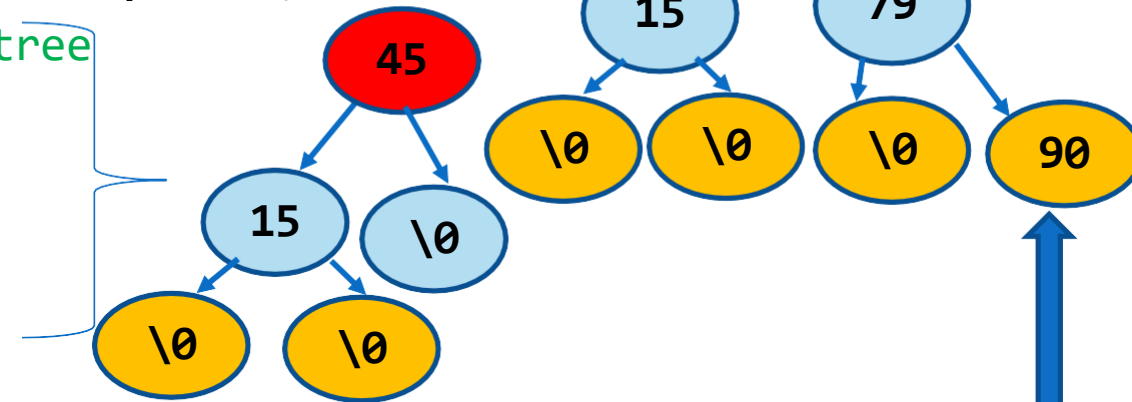
Elements: 45, 15, 79, 90, 10, 55, 12, 50

```
void insert_BST(struct Node *root,int item){  
    if(root == NULL){ //if tree is empty  
        root = newNode(item);  
    }  
    else{  
        current = root;  
        parent = NULL;  
        while(1){  
            parent = current;  
            if(item < parent->data){ //go to left of the tree  
                current = current->leftChild;  
                if(current == NULL){ //insert to the left  
                    parent->leftChild = newNode(item);  
                    return;  
                }  
            }  
            else{ //go to right of the tree  
                current = current->rightChild; //insert to the right  
                if(current == NULL){  
                    parent->rightChild = newNode(item);  
                    return;  
                }  
            }  
        }  
    }  
}
```

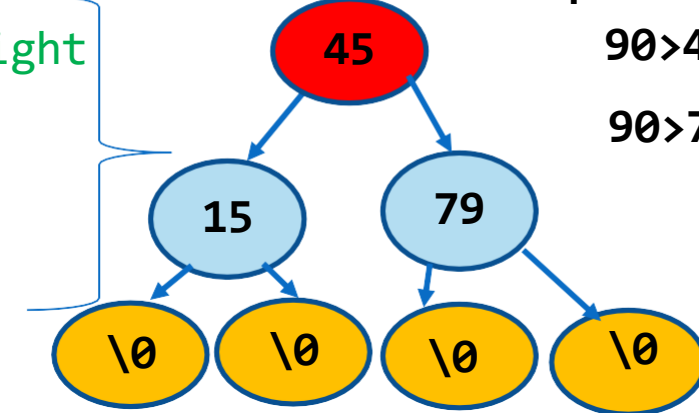
Step-1: 45 root



Step-2: 15, 15 < 45



Step-3: 79, 79 > 45

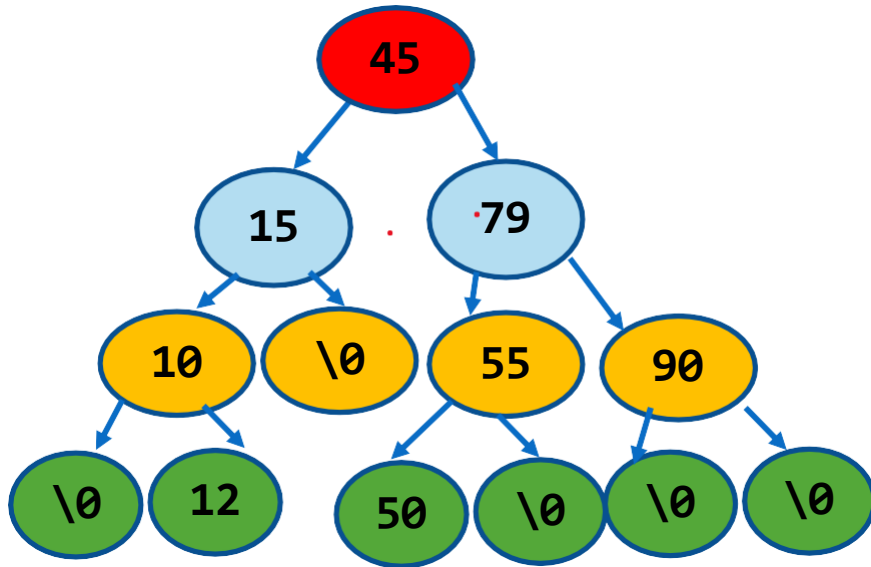


Step-4: 90, 90 > 45

90 > 79



Binary Search Trees (BST)



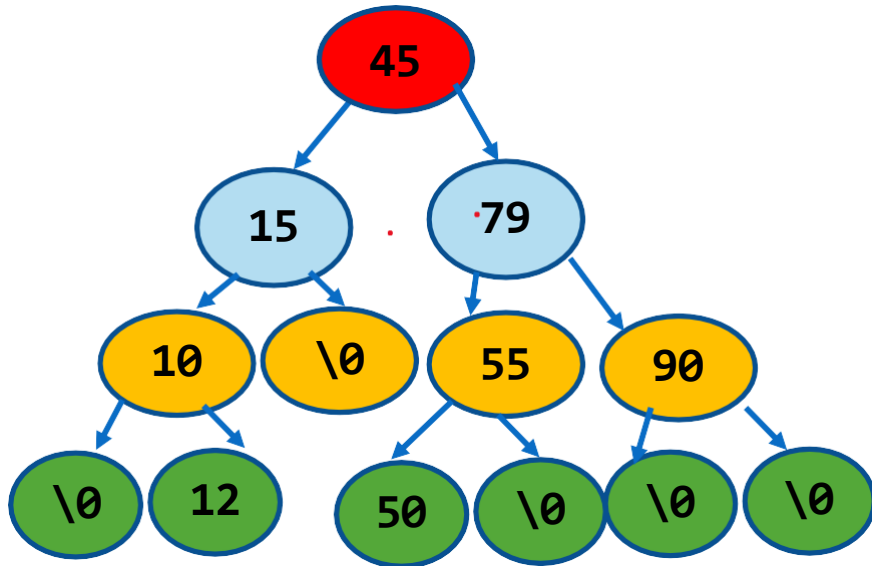
```
void displayBST(struct Node *root)
{
    printf("BST elements are:")
    if(root!=NULL)
    {
        displayBST(root->leftChild);
        printf("%d\n", root->data);
        displayBST(root->rightChild);
    }
}
```

```
struct node *root;
int main()
{
    displayBST(root);
    if(searchBST(root, item)==TRUE)
        printf("Found");
    else
        printf("Not Found");
}
```

```
bool searchBST(struct Node *root, int item)
{
    if(root==NULL)
        return FALSE;
    else if(root->data==item)
        return TRUE;
    else if(item<=root->data)
        searchBST(root->leftChild, item);
    else
        searchBST(root->rightChild, item);
}
```

Binary Search Trees (BST)

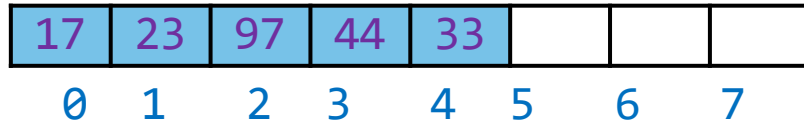
```
void minBST(struct Node *root)
{
    if(root==NULL)
        printf("Empty Tree");
        return -1;
    else if(root->left==NULL)
        return root->data;
    return minBST(root->left);
}
```



```
struct node *root;
int main()
{
    min=minBST(root);
    printf("Min val=%d", min);
    ht=heightBST(root);
    printf("Height=%d", ht);
}
```

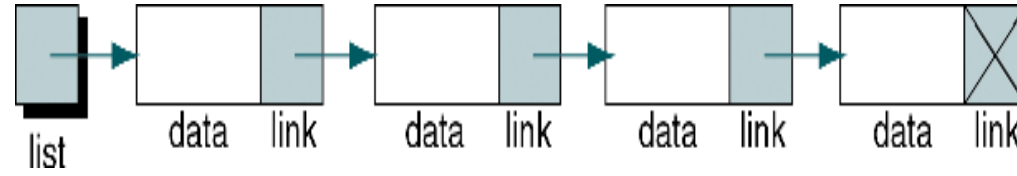
```
void heightBST(struct Node *root)
{
    if(root==NULL)
        return -1;
    else{
        leftHT=heightBST(root->left);
        rightHT=heightBST(root->right);
        if(leftHT>rightHT)
            return(leftHT+1);
        else
            return(rightHT+1);
    }
}
```

Binary Tree Traversal



Traversal in an array:
from $a[0]$ to $a[n-1]$

Linear Traversal



Traversal in a Linked list: from **head** to **NULL**

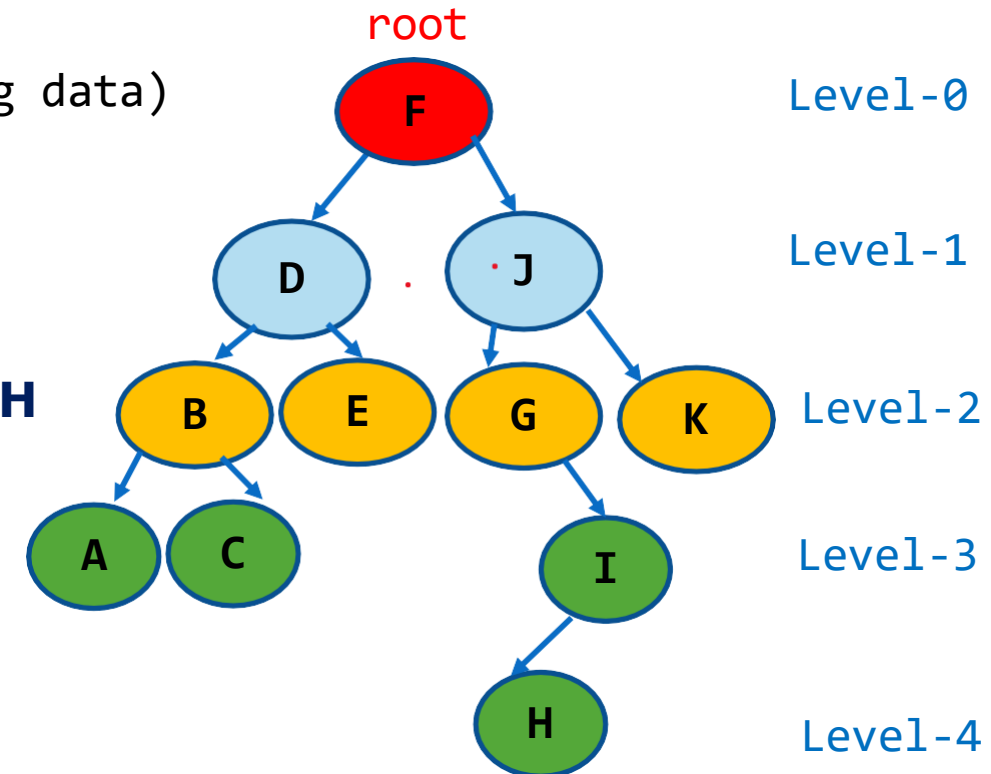
Tree traversal: Process of visiting (reading or processing data) each node in the tree exactly once in some order.

Tree traversal

→ **Breadth First Traversal**
(Level-order traversal): **F, D, J, B, E, G, K, A, C, I, H**

→ **Depth First Traversal**

1. Preorder
<root, left, right>: **F, D, B, A, C, E, J, G, I, H, K**
2. Inorder
<left, root, right>: **A, B, C, D, E, F, G, H, I, J, K**
3. Postorder
<left, right, root>: **A, C, B, E, D, H, I, G, K, J, F**



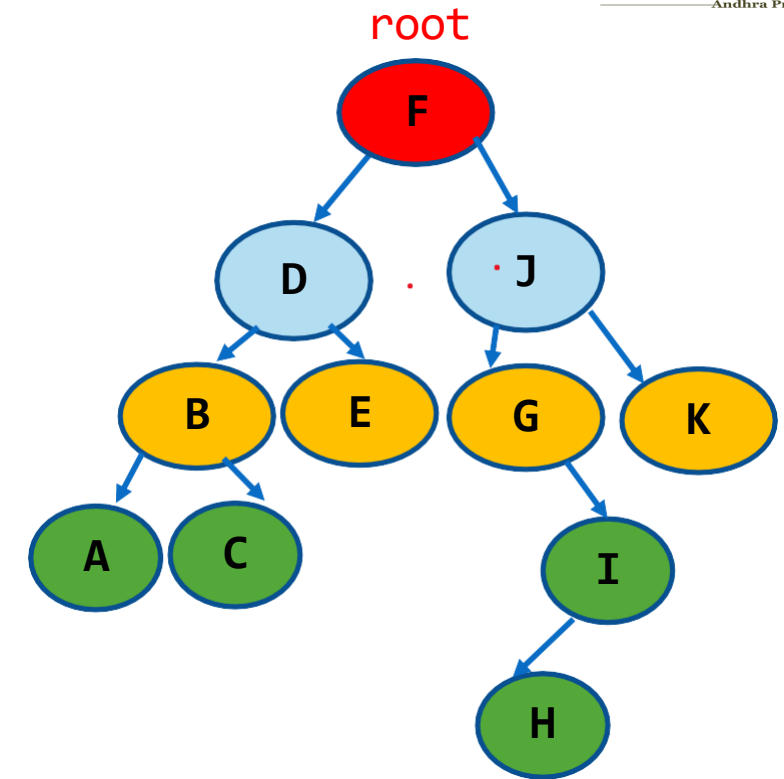
root-→ Print the value,
left-→ Traverse left sub tree,
right-→ Traverse right subtree

Binary Tree Traversal

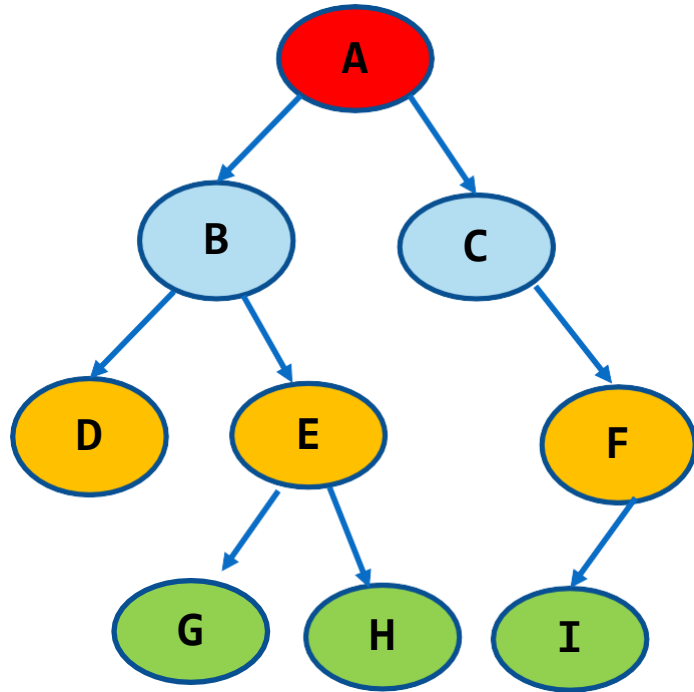
```
void preorder(struct Node *root)
{
    if(root == NULL)
        return;
    printf("%d ",root->data); //visit the node
    preorder(root->left); //traverse the left subtree
    preorder(root->right); //traverse the right subtree
}

void inorder(struct Node *root)
{
    if(root == NULL)
        return;
    inorder(root->left); //traverse the left subtree
    printf("%d ",root->data); //visit the root
    inorder(root->right); //traverse the right subtree
}

void postorder(struct Node *root)
{
    if(root == NULL) return;
    postorder(root->left); //traverse the left subtree
    postorder(root->right); //traverse the right subtree
    printf("%d ",root->data); //visit the root
}
```



Binary Tree Traversal



Preorder:

Inorder:

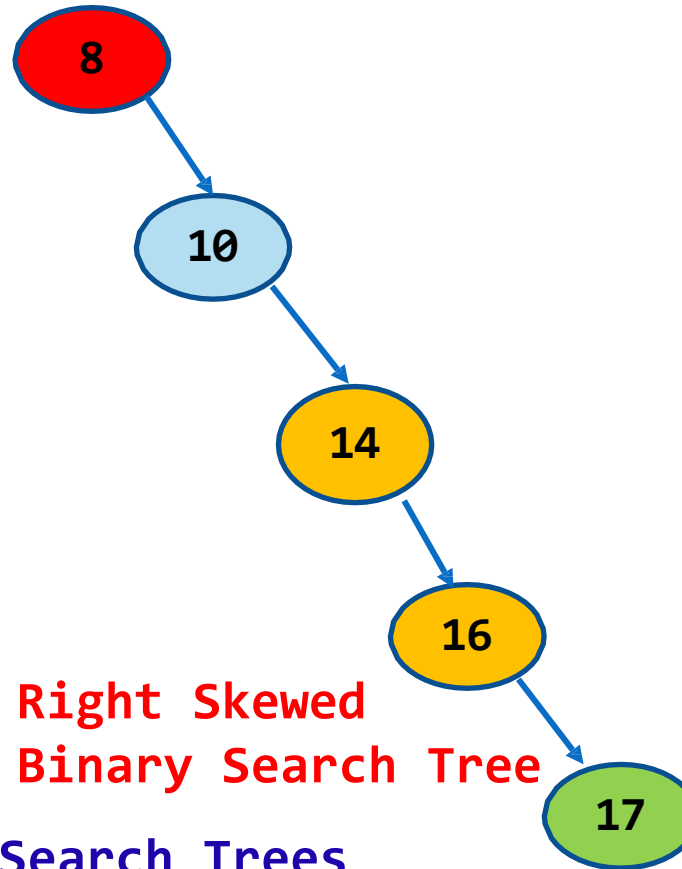
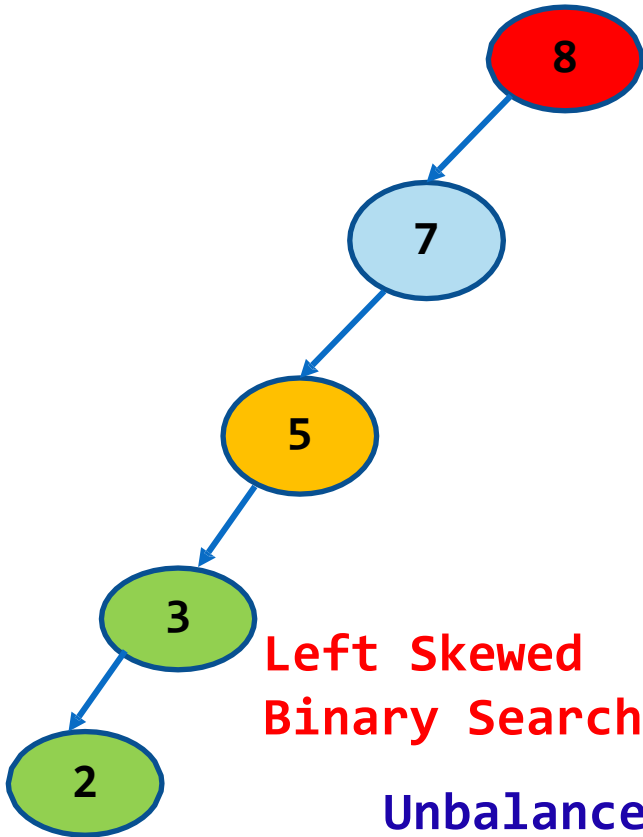
Postorder:

Limitations of BST

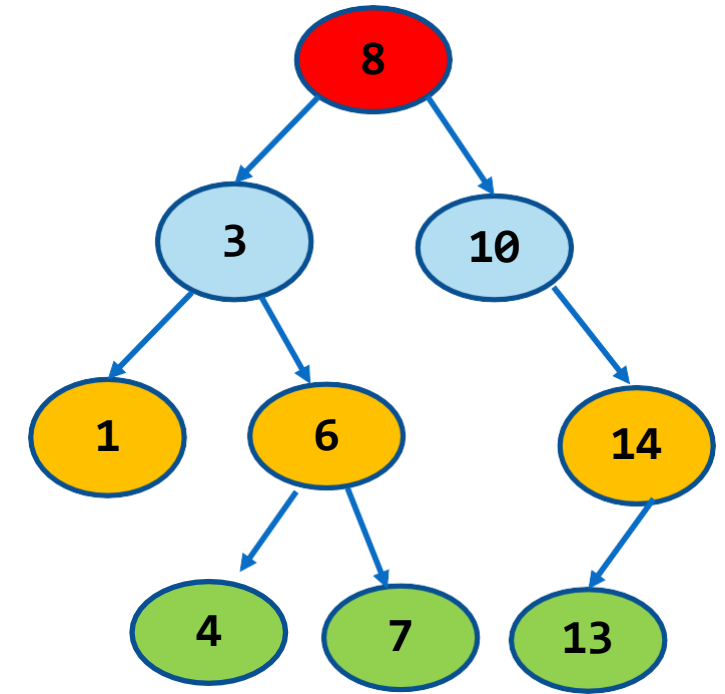
- The minimum height of a BST $\log_2(n)$
- The maximum height of a BST $n-1$
- Want to confirm?
- Then insert the following seven ($n=7$) nodes into a BST:
 - 45, 36, 63, 27, 39, 54, 72.
- What is the height of the BST you got?
- Now, insert the following seven ($n=7$) nodes into a BST:
 - 27, 36, 39, 45, 54, 63, 72.
- Now what is the height of the BST?
- Hope you are now confirmed with first two points.

AVL Trees

Why AVL Trees ?



Unbalanced Binary Search Trees



Balanced Binary Search Tree

Insertion, Deletion or Search operations
take $O(n)$ time in unbalanced BSTs.

Cost can be reduced to $O(\log n)$ with
balanced BSTs.

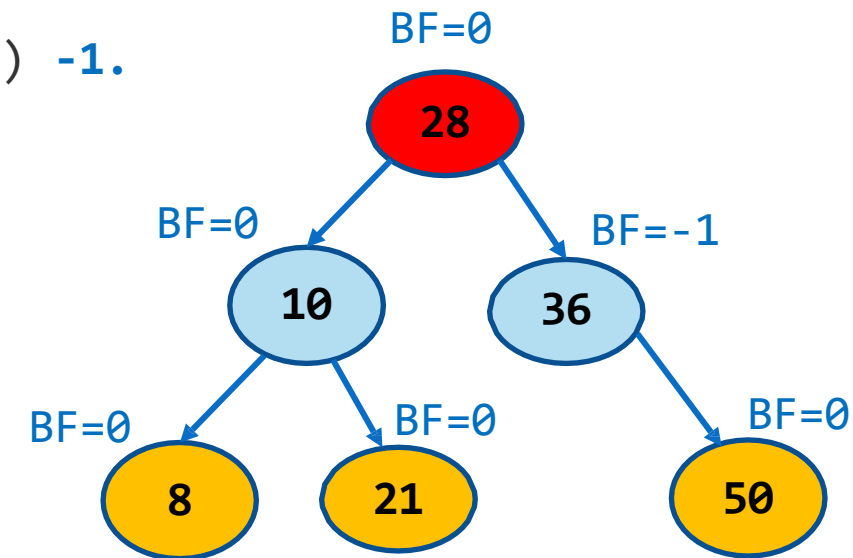
What is AVL Tree (or Height balanced binary search tree) ?

AVL tree is a balanced BST in which the height of left and right subtrees differ by **no more than one**.

- ▶ AVL tree is a BST that is either empty or consists of two AVL subtrees, Left subtree T_L and right subtree T_R whose heights differ by ≤ 1 .
- ▶ The height difference is called **Balance Factor**.
- ▶ Balance factor for any node in AVL tree must be **+1, 0, (or) -1**.

$$H_L - H_R \leq 1$$

AVL tree ensures that Insertion, Deletion and Search operations take **$O(\log n)$** .



AVL Tree

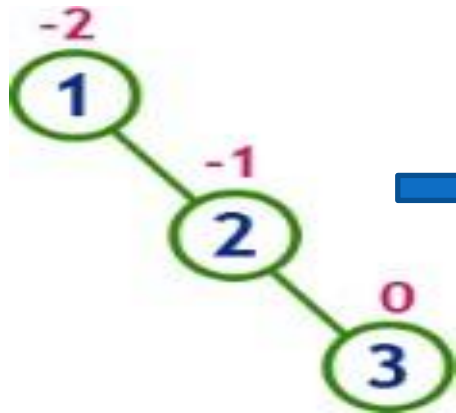
- ▶ In AVL tree, after performing operations like insertion or deletion, the **balance factor** of every node in the tree should be maintained.
- ▶ If tree becomes imbalanced due to any operation, the **rotation** operations are performed to make it balanced.

- | | |
|--|-------------------|
| 1) L L rotation : Inserted node is in the left subtree of left subtree of A | } Single rotation |
| 2) R R rotation : Inserted node is in the right subtree of right subtree of A | |
| 3) L R rotation : Inserted node is in the right subtree of left subtree of A | } Double rotation |
| 4) R L rotation : Inserted node is in the left subtree of right subtree of A | |

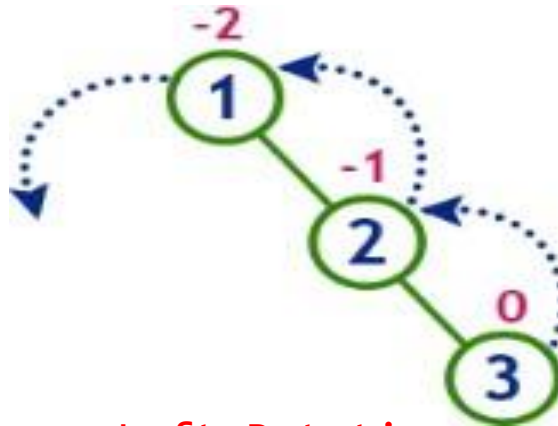
Here node A is the **critical node (closest ancestor)** whose balance Factor is other than **-1, 0, 1**

AVL Tree Rotations

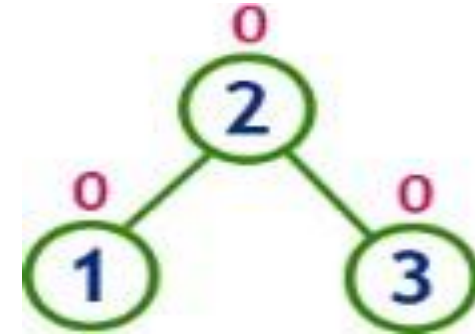
1. R R Rotation (Single Left Rotation)



Imbalanced AVL tree

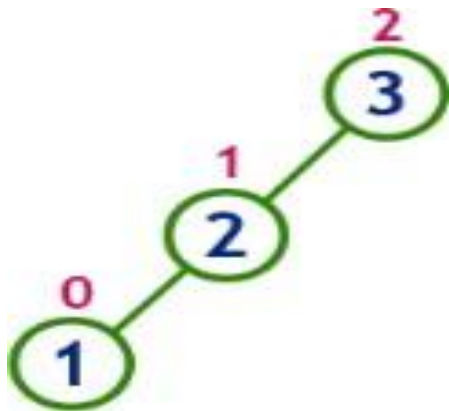


Left Rotation

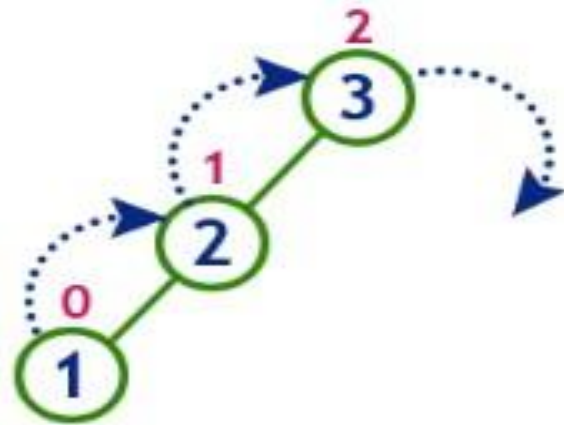


Balanced AVL tree after rotation

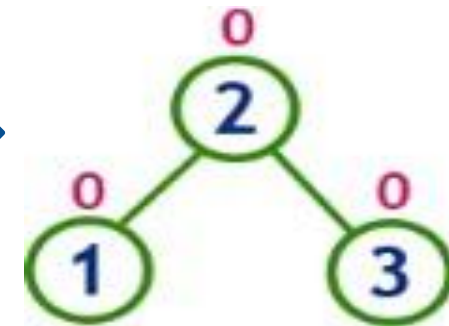
2. L L Rotation (Single Right Rotation)



Imbalanced AVL tree



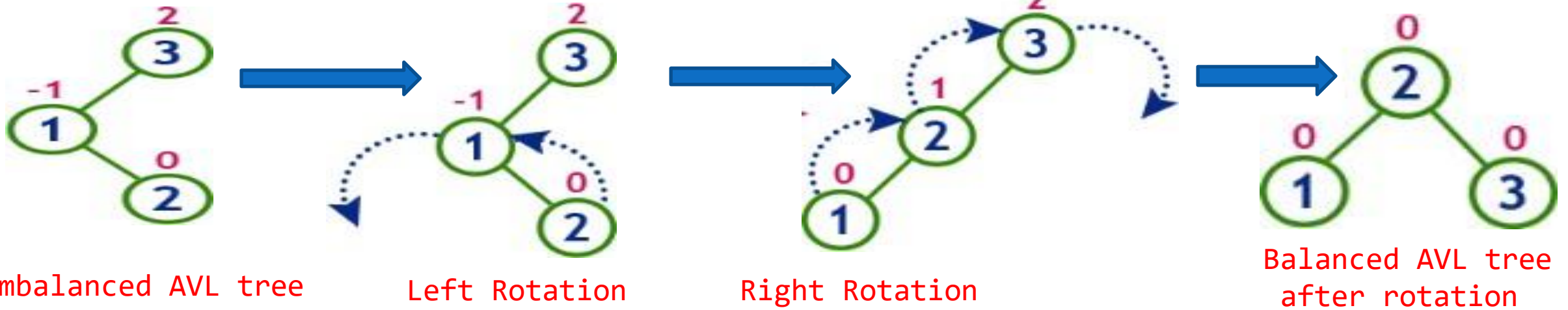
Right Rotation



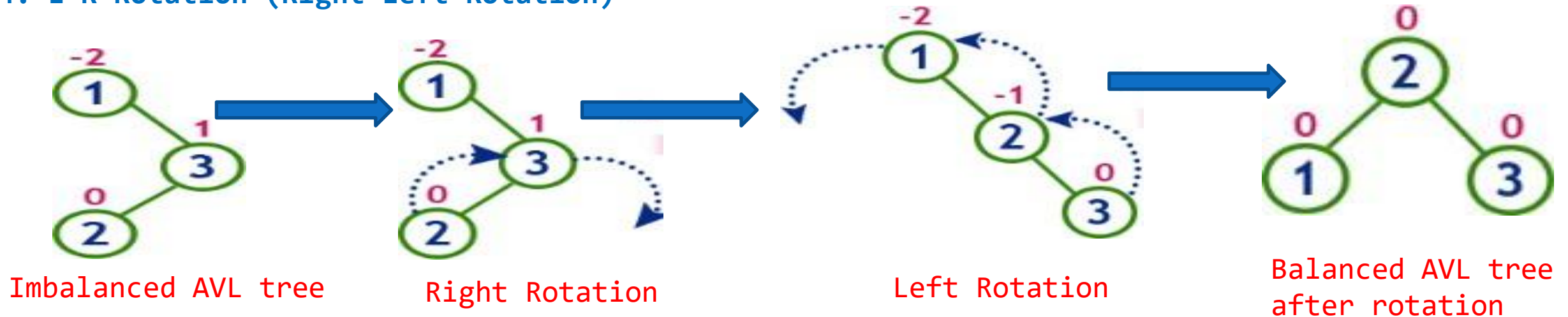
Balanced AVL tree after rotation

AVL Tree Rotations

3. R L Rotation (Left Right Rotation)

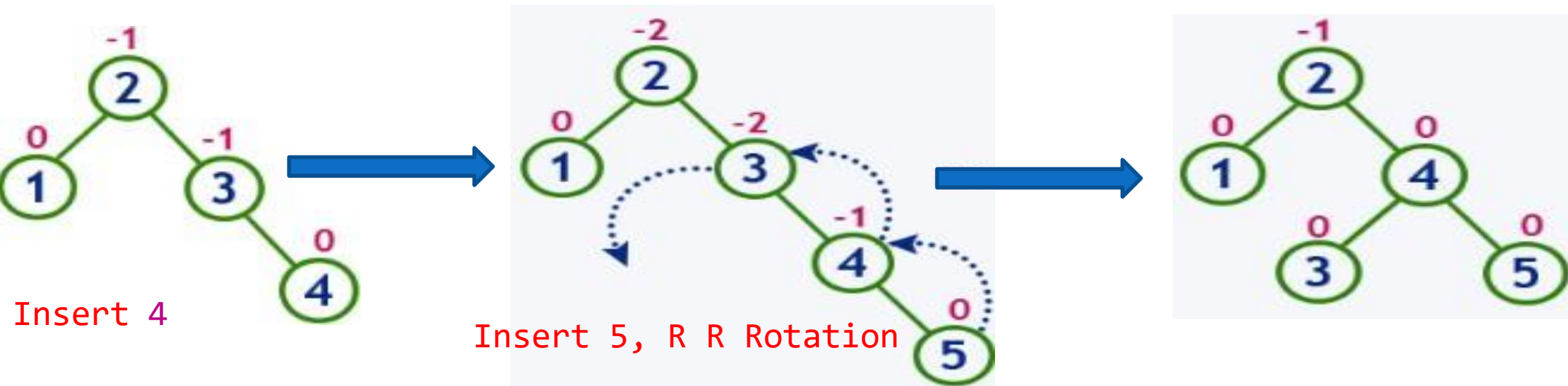
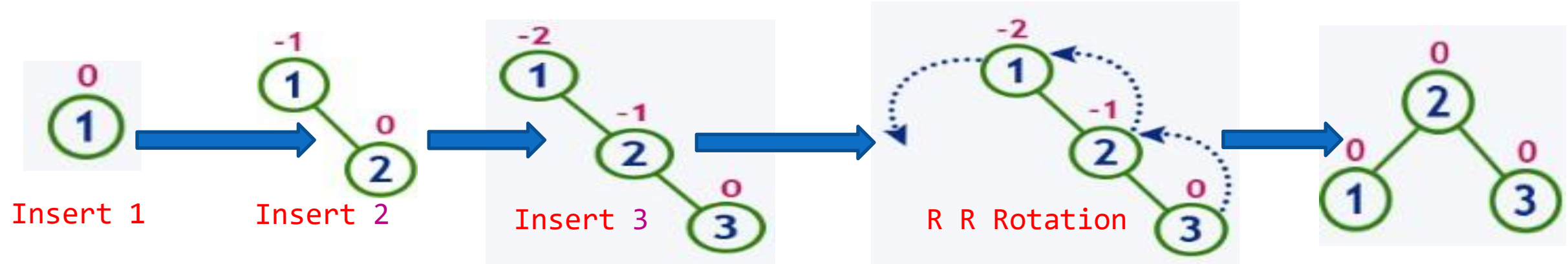


4. L R Rotation (Right Left Rotation)

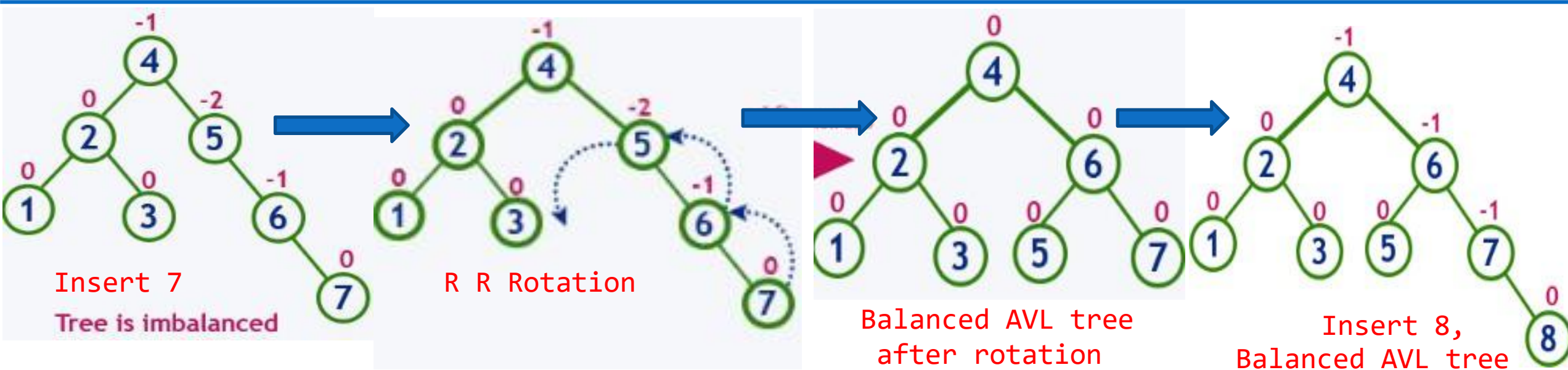
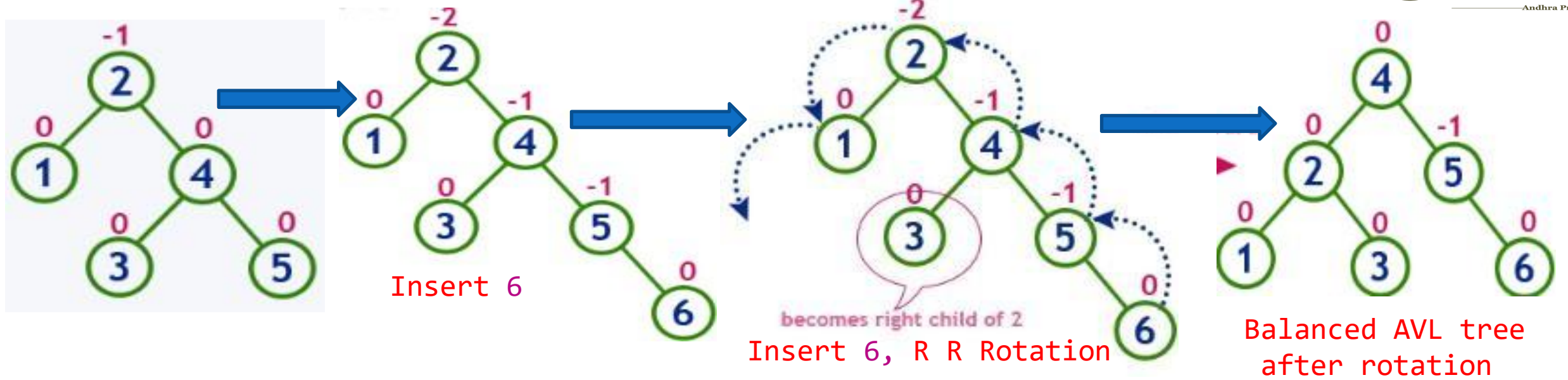


Insertion on AVL Tree

Insert 1 to 8 numbers into AVL Tree



Insertion on AVL Tree



AVL Trees - Deletion

- Deletion in AVL tree follows the same rules as the deletion in BST.
- However, this time the balance of the AVL tree is checked. If it is imbalanced the rebalancing is necessary
- **Deletion cases:**
 1. Case1: If the node to be deleted has no children, i.e., it is a leaf node, then you are done. But the tree must be balanced after updates.
 2. Case2: If the node to be deleted has either left or right sub-tree (LST or RST), then link that ST with the parent of the deleted node.
 1. Case3: If the node to be deleted has both sub-trees then replace the position of the deleted node with
 - a. either the largest node of LST
 - b. or smallest node of the RST

AVL Trees - Deletion

- Once the deletion and linking is done, update the BFs.
- If the tree is imbalanced, then do rebalancing.

▪ Rebalancing cases:

1. $R_0 \rightarrow$ LL rotation

2. $R_{+1} \rightarrow$ LL rotation

3. $R_{-1} \rightarrow$ LR rotation

4. $L_0 \rightarrow$ RR rotation

5. $L_{+1} \rightarrow$ RL rotation

6. $L_{-1} \rightarrow$ RR rotation

- How to interpret those 6 cases?

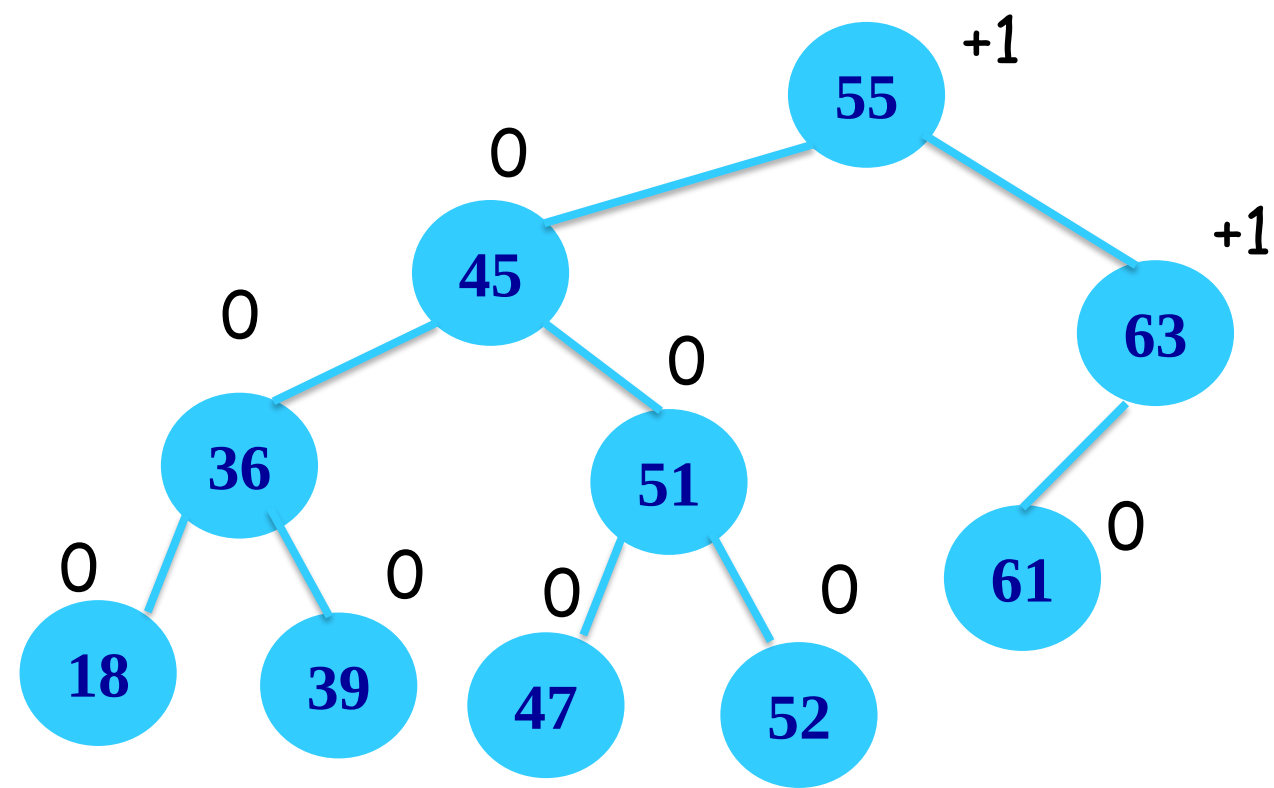
Steps:

- After updating the BFs, find the critical node (CN).
- Now determine the BF of the left child, for instance, of CN. Let us say its BF = -1
- Suppose the deleted node, for instance, is to the right of the CN ,then choose L (for left)
- Now put together, read it as L-1 case. Implies RR rotation must be performed.

AVL Trees - Deletion

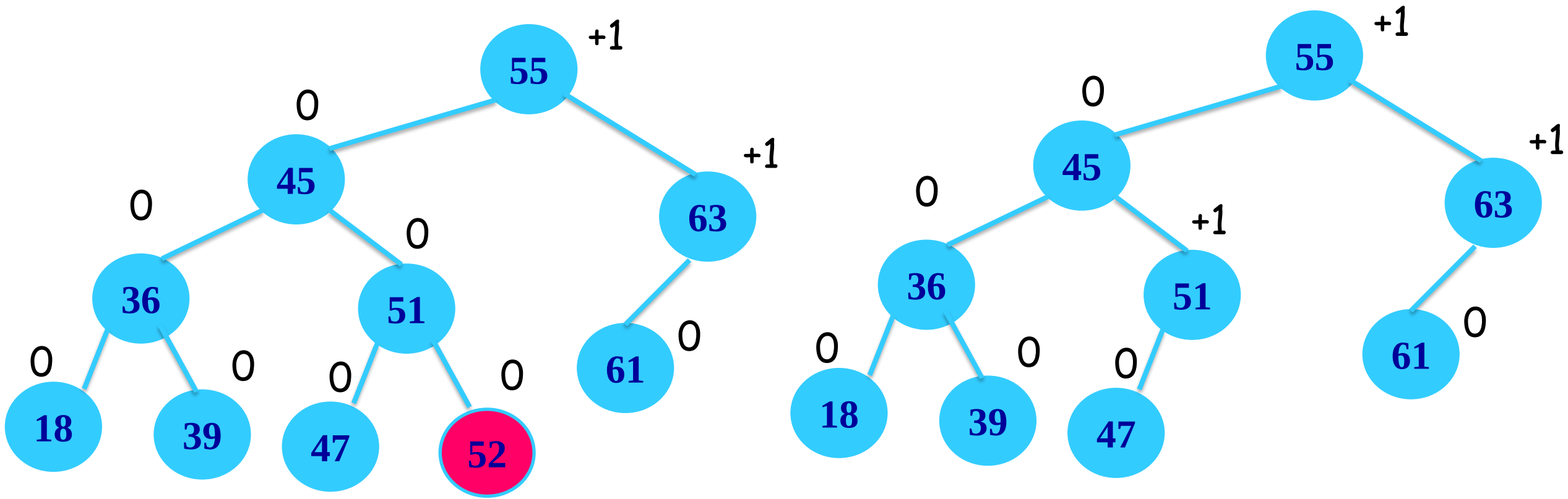
Examples:

In the given AVL tree, delete the following nodes: 52, 36 and 61.



AVL Trees - Deletion

Since 52 has no children, we just need to update the BF and determine the BF.



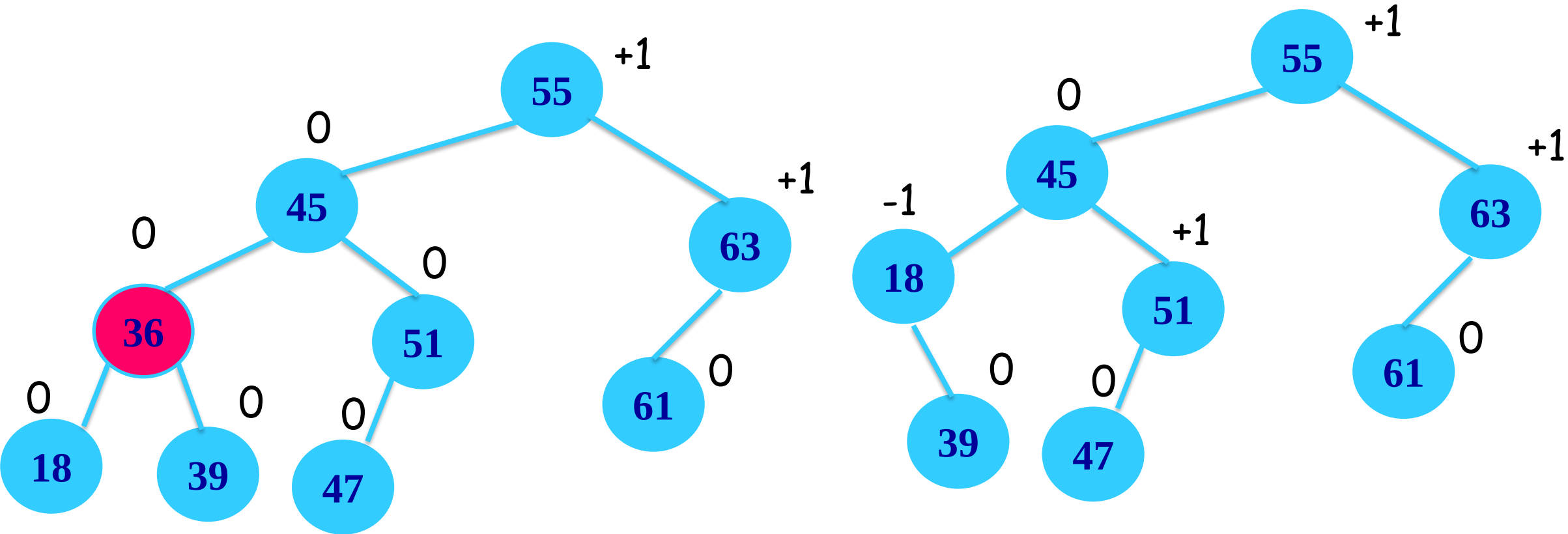
- Node 52 has no children, so we just need to update the BF and determine the balancing criteria.
- The updated BFs indicate that the tree is balanced and no rebalancing is necessary.

AVL Trees - Deletion

Delete 36:

Since 36 has two children, its position can be replaced by either largest node in LST or smallest node in RST.

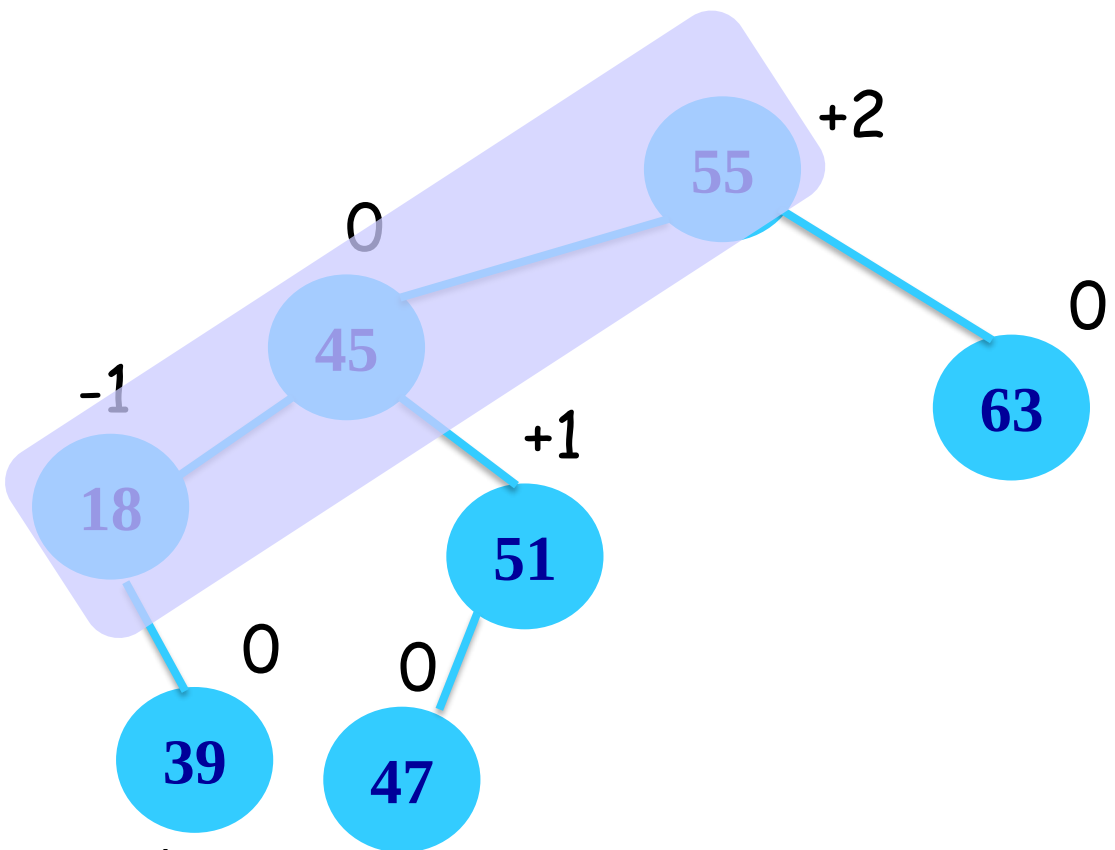
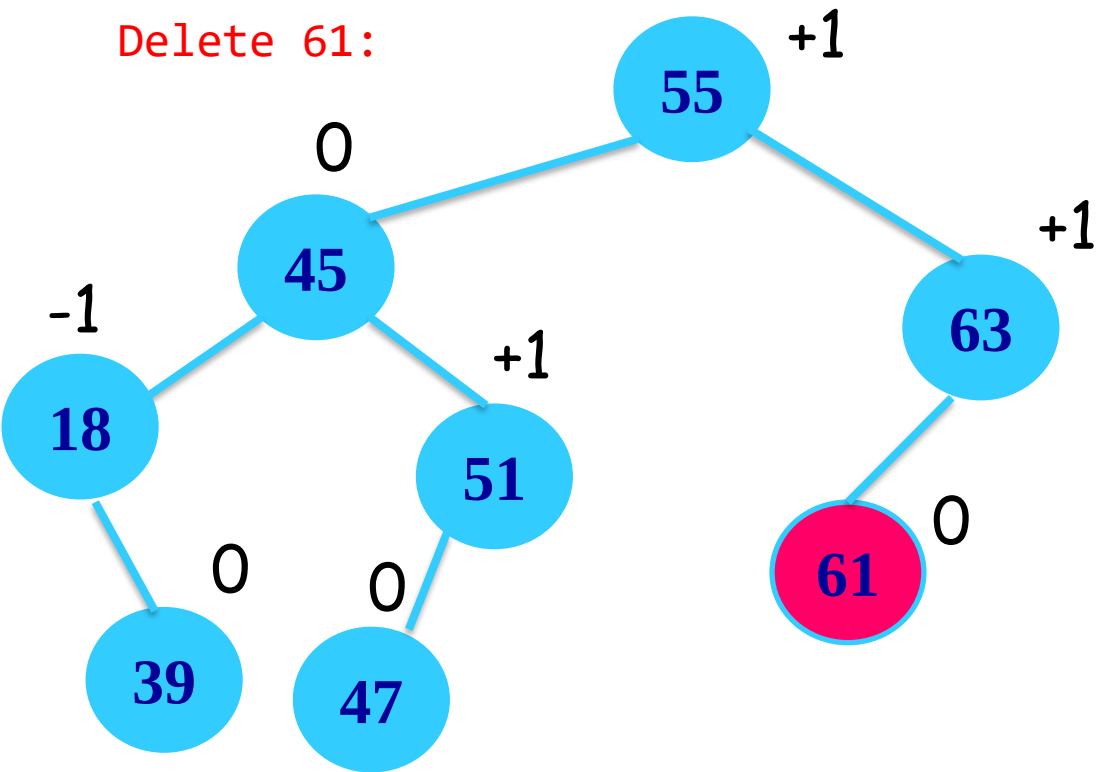
Let us choose largest node in LST



- The updated BF's indicate that the tree is balanced and no rebalancing is necessary.

AVL Trees - Deletion

Delete 61:

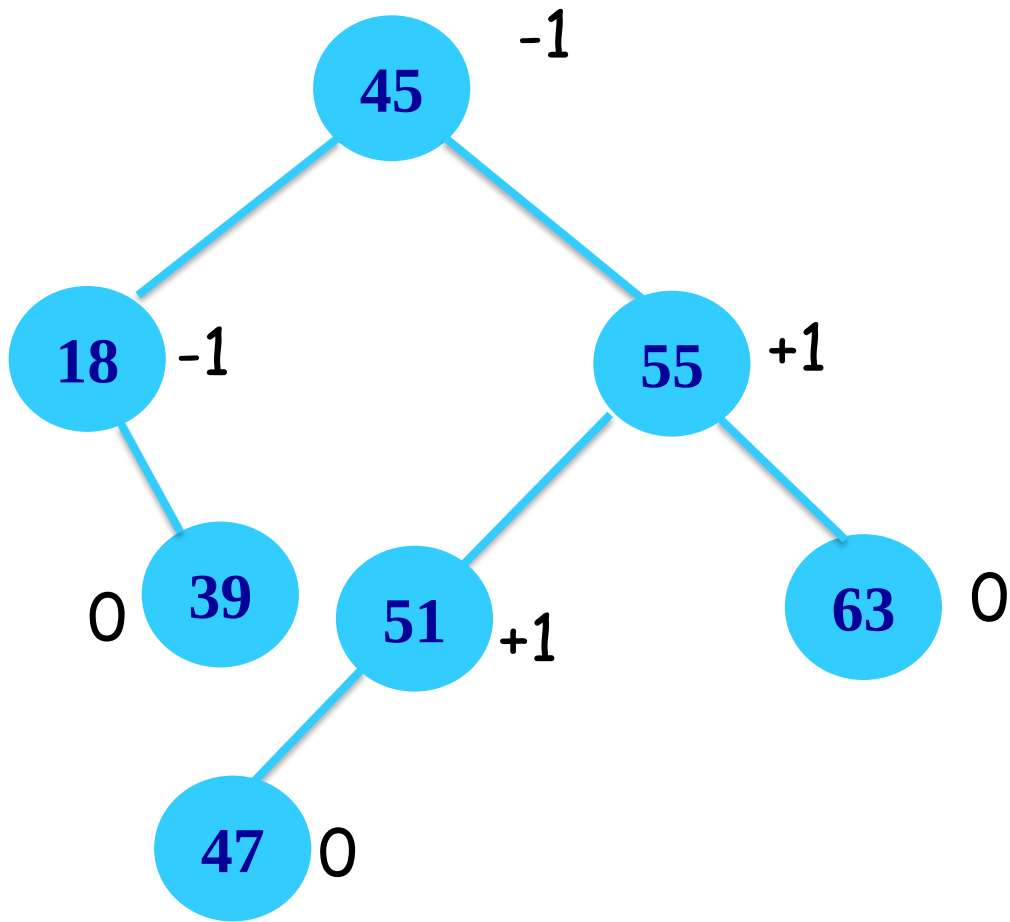


- After updating the BF's, we find that node 55 becomes the CN.
- Now check for the rule patterns as follows:
 - Node deleted was on to the right of CN, so it is R.
 - The left child of CN has BF = 0 (i.e., node 45's BF is 0), so the combination is R0.
 - So perform LL rotation.

Note: For rotations we need 3 nodes. Starting from node CN and downwards towards the leaf, choose 3 nodes that are in LL pattern.

AVL Trees - Deletion

After rotations the tree is as shown below with BF updated.



Expression Trees

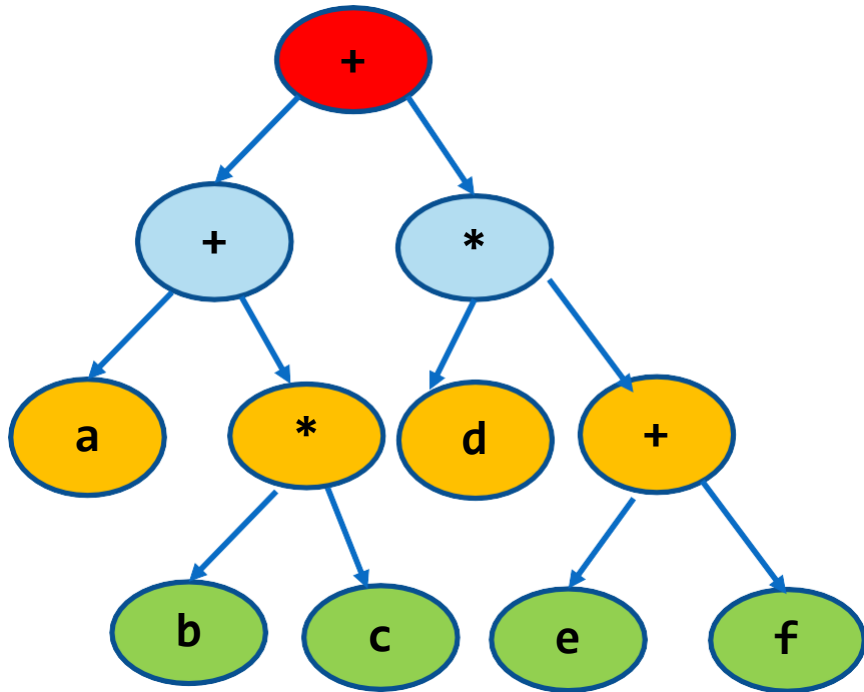
It is a binary tree that represents an expression with the following properties:

- Each leaf is an operand. Ex: **a, b, c, 5, 10**
- The root and internal nodes are operators. Ex: **+, -, *, /, ^**
- Subtrees are subexpressions with the root being an operator.

Example expression:

a + (b * c) + d * (e + f)

Equivalent Expression Tree:



An **infix expression** can be produced by performing **in-order traversal** on expression tree

((a+(b*c))+(d*(e + f)))

A **prefix expression** can be produced by performing **pre-order traversal** on expression tree

++ a * b c * d + e f

A **postfix expression** can be produced by performing **post-order traversal** on expression tree

a b c * + d e f + * +

Constructing Expression tree from postfix expression

Step-1: Read one symbol at a time from the postfix expression.

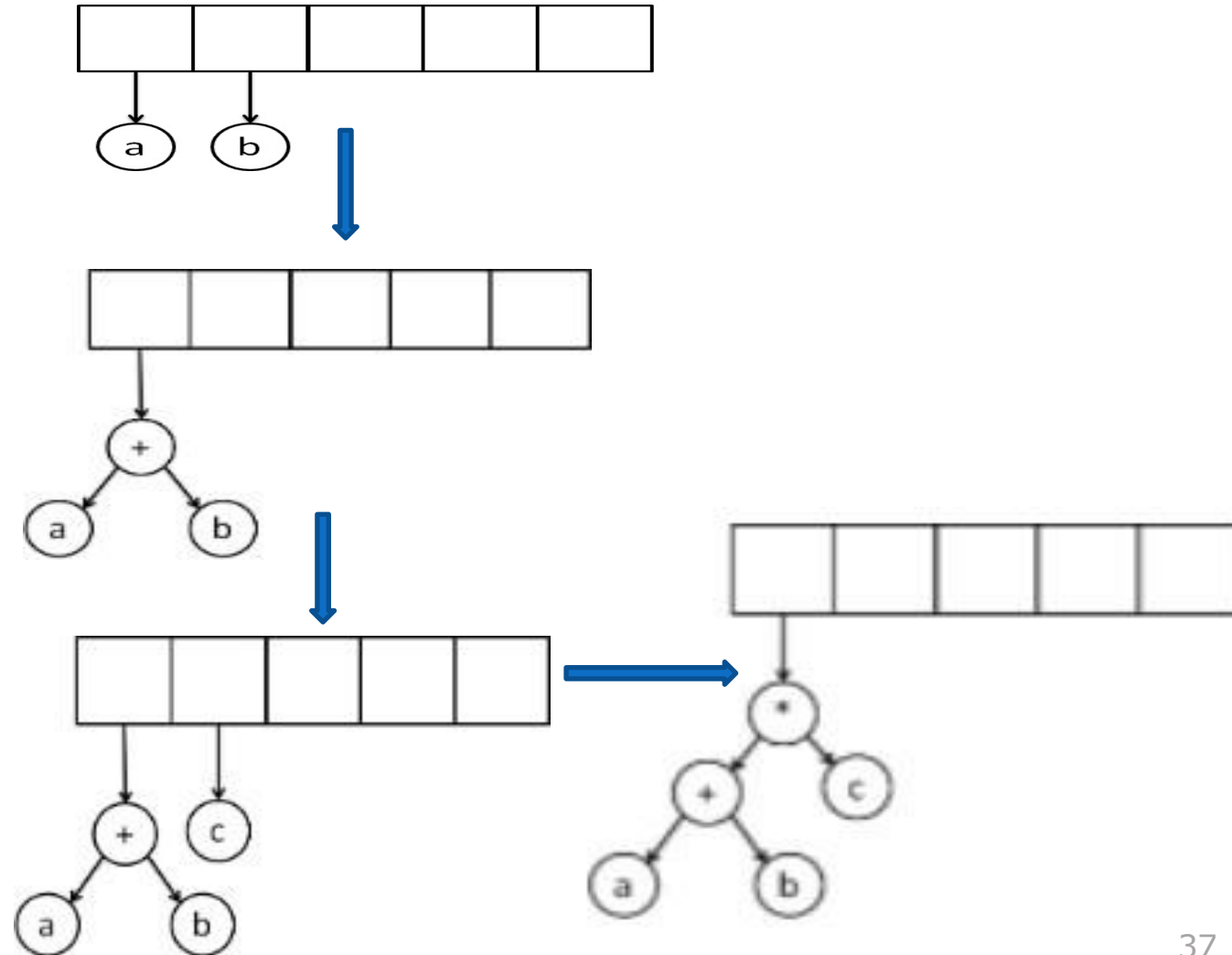
Step-2: Check if the symbol is an operand or operator.

Step-3: If the symbol is an operand, create a one node tree and push a pointer onto a stack

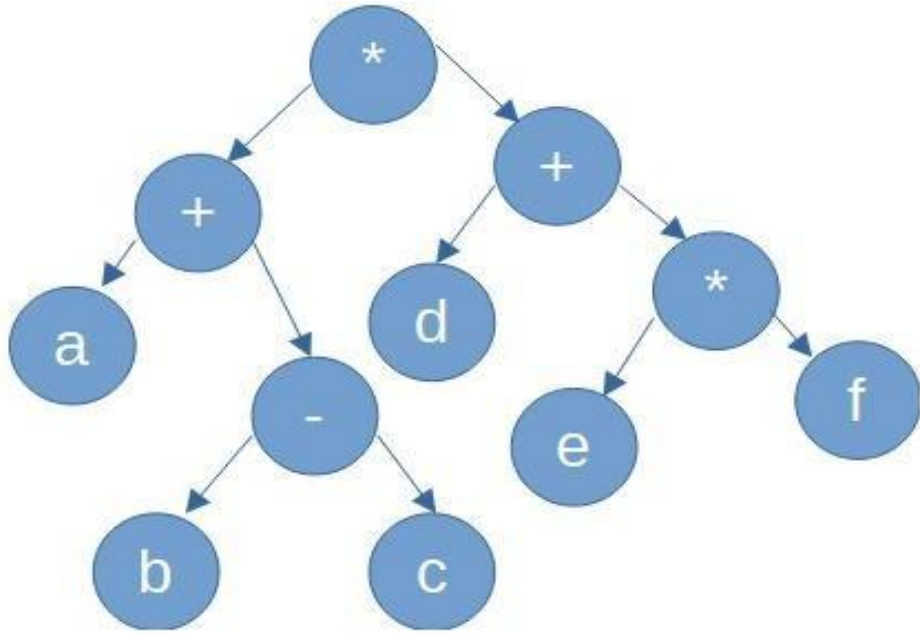
Step-4: If the symbol is an operator, pop two pointers from the stack namely T_1 & T_2 and form a new tree with root as the operator, T_1 & T_2 as a left and right child

Step-5: A pointer to this new tree is pushed onto the stack

Example: $a \ b \ + \ c \ *$



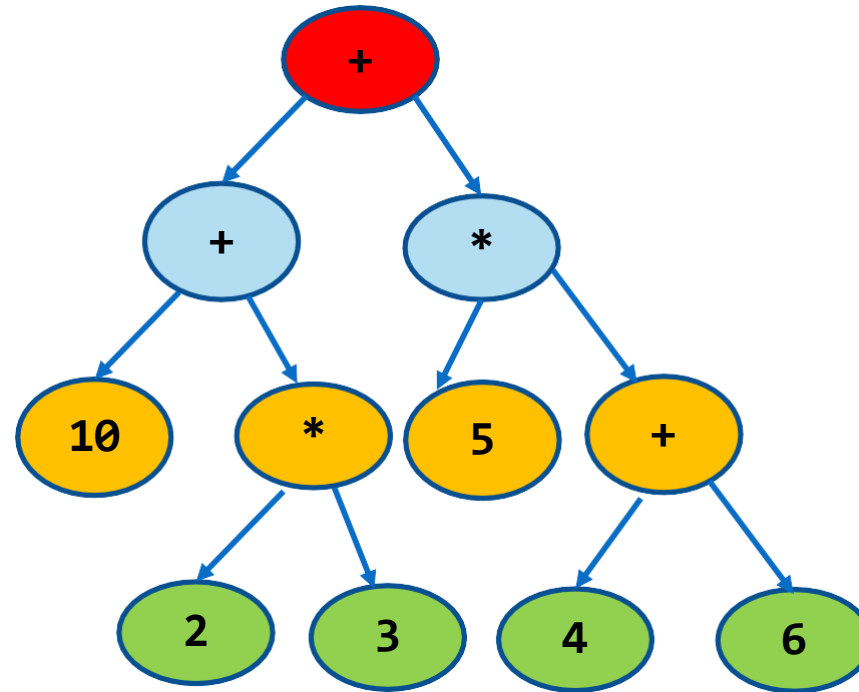
Expression Trees



Infix expression:

Prefix expression:

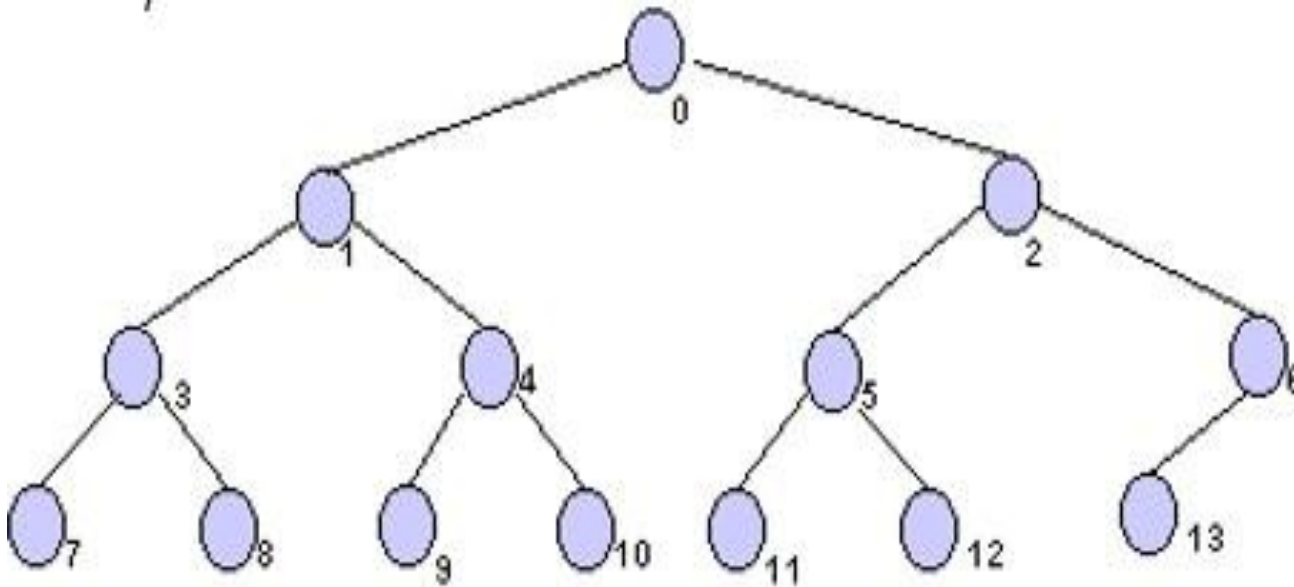
Postfix expression:



Postfix expression evaluation:

Array Representation of Complete Binary Tree

Tree Structure: **Array** indices carry tree structure



Parent of $a[k] = a[(k-1)/2]$

Left child of $a[k] = a[2*k+1]$

Right child of $a[k] = a[2*k+2]$

Array **index order** = level order (beginning with 0)

If parent node is at index k in the array then

left child of that node is at index $(2*k + 1)$

right child is at index $(2*k+ 2)$ in the array.